BusyLean: Naming Ever Larger Numbers in Lean

Research notes on constructive large numbers with formal comparisons

BusyLean contributors

October 2026

Abstract

We port Cody Roux's *name the biggest number* contest from Coq to Lean 4. In the contest every number must be a closed, computable definition, and every claim "my number is bigger" must be a kernel-checked theorem. We give an exact port of the incumbent Contender 5 (the largest numeral of a raw System T term of depth 42). We review and rebuild five submitted proposals, and we construct Contender 6, which combines their best ideas: an object language with W-types over every finite type, relativized to oracle families, and reflected along every tree that the language itself can define. Along the way we prove formally that one proposal's re-implementation of the old language agrees with the exact port. We then construct Contender 7: System F with Contender 6 as an oracle. Its strong normalization is proved inside Lean by Girard's reducibility candidates, which is possible because Lean's `Prop` is impredicative, and normalization runs as a certified total function. Contender 8 adds type operators (System F_ω), with candidates lifted to kinds and a sound, fuel-bounded type checker. Contender 9 reflects F_ω along the W-language's trees, and we explain why the Calculus of Constructions is not a growth-rate jump. Contender 10 makes the jump cheaply: kind polymorphism lets one program reach kinds of unbounded order at run time, beyond higher-order arithmetic. A second batch of five proposals is reviewed; two of its ideas, operator-level reflection and ordinal notations above a language-defined index, are adopted in Contender 12. Contender 13 adds a predicative universe of operators, a kind `dyn` interpreted as $\sum_k \mathrm{Sem}(k)$, with level-1 kind variables that range over it. Contender 14 climbs a ladder of such universes: at input n it reflects over the language with n stacked universes, all normalized by one theorem that is parametric in the universe. A fourth batch of proposals contributes a rank hierarchy of transformer lifts, which Contender 15 applies to the ladder. Contender 16 makes the ladder transfinite: Lean builds a universe for every well-founded atom order, and Cantor normal forms give a ladder of length ω^ω. Contender 17 extends the ladder to length ε_0 by applying the same well-foundedness lemma hereditarily, and Contender 18 to length Γ_0 with a Schütte-style proof for binary Veblen terms that follows only the comparison's case split. Contender 19 extends the same technique to Schütte's Klammersymbole, whose argument positions are themselves terms, giving a ladder of large-Veblen length. Contender 20 reaches the Bachmann–Howard ordinal with a Buchholz-style collapsing notation, whose well-foundedness proof is the classical impredicative argument carried by Lean's `Acc`. Contender 21 bounds the number of collapsing levels, which turns the multi-level argument into a downward induction over relativized scopes. It then diagonalizes over the bound, so ladder lengths climb to Buchholz's ordinal $\psi_0(\Omega_\omega)$. Contender 22 relativizes the bounded systems to a well-founded base and iterates, which goes beyond $\psi_0(\Omega_\omega)$. Contender 23 adds a proposal's transfinite-rank hierarchy of lifts, and Contender 24 iterates the towers along well-founded indices, each round indexed by the previous one. Contender 25 adopts a proposal's key idea: atoms are addresses in a Lean-defined tree, so well-foundedness is structural and free. The ladder then runs over trees built by Buchholz-style collapsing functions defined by plain structural recursion, heuristically to $\psi_0(\varepsilon_{\Omega_\omega+n})$. Contender 26 indexes the collapsing levels by tree ordinals themselves, heuristically climbing toward the fixed points of $\alpha \mapsto \psi_0(\Omega_\alpha)$. Contender 27 stratifies level codes into universes, toward an inaccessible-like level, and Contender 28 stacks

the universes along tree ordinals given by earlier collapses. Contender 29 iterates the universe-building operators themselves, indexed by tree ordinals, and Contender 30 lifts closure to all finite types over universes; Contender 31 to transfinite types indexed by collapsed trees. Contender 32 turns the constructions into programs of a typed language over tree ordinals with a semantic interpreter, and takes all programs of bounded size. Contender 33 stacks the language on itself, and Contender 34 stacks it along tree ordinals. Contender 35 takes its trees from impredicative System F programs, made well-founded by a reducibility candidate for trees. Contender 36 lets the ladder language build the trees, so each ladder defines the length of the next. Contender 37 stacks the tree languages over this jump. A correction (Section 38): a ladder program of depth n sees only a finite truncation of its atom order, so Contenders 15–37 grew through their amplifiers, not through the size of their atoms. Contender 38 runs a fast-growing hierarchy along the trees themselves, and Contender 39 interleaves the impredicative jump with predicative collapse. System FT2 adds tree recursion over an input tree, which makes the jump a genuine operator, and Contender 40 places it under a metalanguage with first-class language closure. Contender 41 gives the ladder language the same recursor over its input tree, and Contender 42 collapses relative to these jumps, and Contender 43 makes the collapse a first-class closure inside the metalanguages. Contender 44 gives that closure collapsing levels indexed by trees. Contender 45 makes each stage of the construction the base primitive of the next, iterated along a tree. Contender 46 collapses the universes of codes of Contenders 27–31 relative to the closure's operator. Contender 47 iterates that self-based iteration itself. Contender 48 indexes such iteration levels by a tree. Contender 49 reads its final tree with impredicative programs instead of a fixed hierarchy. Contender 50 reads it with numeric ladder programs. Contender 51 makes the iteration operators primitives of a language over trees. Contender 52 stacks those languages along a tree. Contender 53 iterates the reflection along the tree, each level an oracle of the next. Contender 54 makes that reflection a primitive of the language, applicable to any tree a program builds. Contender 55 stacks the number languages along the tree, each level a first-class closure for the next. Contender 56 stacks the tree and number languages jointly. Contender 57 lets programs choose the stacking heights. Contender 58 repeats that step to a tower of N stack-reflecting languages. Contender 59 indexes that tower by a tree. Contender 60 iterates the tower operator itself along a tree. Contender 61 makes the tower operators values of a language with products. Contender 62 stacks that language along a tree. Contender 63 makes its stacking operator a value of the language. Contender 64 towers that step along a tree. Contender 65 makes the pattern of Contenders 57–64 a hierarchy of step operators along a tree. Contender 66 makes closing and towering values, so programs build such hierarchies themselves. Contender 67 raises the closing operator itself along a tree. Contender 68 makes that tower's step a value. Contender 69 raises the step language's own closure operator along a tree. Contender 70 repeats this reflection cycle N times.

Contents

1 The game **6**

2 How to compare unimaginably large numbers **6**

3 Contender 5, exactly **7**

4 Five proposals **7**

5 Contender 6 **8**

5.1 The object language 8

5.2 Oracles for free 8

5.3 Reflection along the language's own trees 8

5.4 What is proved . . . 8

6 Engineering lessons **9**

7 Contender 7: impredicative polymorphism **9**
7.1 Why Lean can host System F . . . 9
7.2 The formalization . . . 9
7.3 From programs to numbers . . . 10
7.4 The contender . . . 10

8 Contender 8: type operators **10**
8.1 Candidates over kinds . . . 11
8.2 Deciding type equality, soundly . . . 11
8.3 The contender . . . 11

9 Contender 9: reflecting F_ω along the W-language's trees **11**
9.1 Why not the Calculus of Constructions . . . 11
9.2 A multiplicative step . . . 12
9.3 Budgets for a growing stack . . . 12

10 Contender 10: kind polymorphism **12**
10.1 Unbounded orders without dependent types . . . 12
10.2 The model . . . 13
10.3 The contender . . . 13

11 Contender 11: reflecting the kind-polymorphic language **13**

12 A second batch of proposals, and Contender 12 **13**
12.1 Five more proposals . . . 13
12.2 Reusing the generic ideas . . . 14

13 Contender 13: a predicative universe of operators **14**
13.1 The idea . . . 14
13.2 The model . . . 15
13.3 The contender . . . 15
13.4 A third batch of proposals . . . 15

14 Contender 14: the ladder of universes **15**
14.1 The idea . . . 15
14.2 One normalization proof for every level . . . 16
14.3 The contender . . . 16

15 A fourth batch of proposals, and Contender 15 **17**
15.1 Four proposals over Contender 13 . . . 17
15.2 Contender 15 . . . 17

16 Contender 16: a transfinite ladder **17**
16.1 The length of the ladder is the lever . . . 17
16.2 Well-founded Cantor normal forms . . . 18
16.3 The contender . . . 18

17 Contender 17: a ladder of length ε_0 **18**

18 Contender 18: a ladder of length Γ_0 **19**
18.1 Veblen terms and a case-split proof . . . 19
18.2 The contender . . . 20

19 Contender 19: a ladder of large-Veblen length **20**
19.1 Klammersymbole . . . 20
19.2 Well-foundedness . . . 20
19.3 The contender . . . 20

20 Contender 20: a ladder of Bachmann–Howard length **21**
20.1 Collapsing, and why the case split no longer suffices . . . 21
20.2 The impredicative argument in Lean . . . 21
20.3 The contender . . . 21

21 Contender 21: Buchholz ladders up to $\psi_0(\Omega_\omega)$ **21**
21.1 Why more levels are hard . . . 21
21.2 A level bound turns it into a downward induction . . . 22
21.3 The contender . . . 22

22 Contender 22: a tower of relativized Buchholz notations **22**
22.1 The barrier at $\psi_0(\Omega_\omega)$ . . . 22
22.2 Relativizing to a base . . . 23
22.3 The contender . . . 23

23 Proposal P22 and Contender 23: transfinite ranks **23**

24 Contender 24: towers along well-founded indices **24**

25 Contender 25: well-foundedness for free **24**
25.1 More proposals, and one decisive idea . . . 24
25.2 Addresses . . . 25
25.3 Collapsing functions by structural recursion . . . 25
25.4 The contender . . . 25

26 Contender 26: transfinitely many levels **25**

27 Contender 27: universes of level codes **26**

28 Contender 28: universes along tree ordinals **26**

29 Contender 29: higher-order universe operators **27**

30 Contender 30: closure functionals of all finite types **27**

31 Contender 31: closure functionals of transfinite type **27**

32 Contender 32: a programming language for tree ordinals **28**

33 Contender 33: languages stacked on languages **28**

34 Contender 34: languages stacked along tree ordinals **28**

35 Contender 35: impredicative trees **29**

36 Contender 36: ladders that build their own length **29**

37 Contender 37: languages stacked over the ladder jump **30**

38 A correction: what a ladder sees **30**

39 Contender 38: a fast-growing hierarchy along the trees **31**

40 Contender 39: the jump mixed with collapse **31**

41 System FT2: an impredicative jump that reads its input **31**

42 Contender 40: the jump under first-class language closure **32**

43 Contender 41: the ladder jump that reads its input **32**

44 Contender 42: collapsing relative to the meta-jump **33**

45 Contender 43: collapse as a first-class closure **33**

46 Contender 44: collapsing closures with tree-indexed levels **33**

47 Contender 45: the construction as its own jump **34**

48 Contender 46: universes collapsed relative to the closure's operator **34**

49 Contender 47: iterating the self-based iteration **34**

50 Contender 48: a tree-indexed hierarchy of iteration levels **35**

51 Contender 49: impredicative reflection over the tree **35**

52 Contender 50: ladder reflection over the tree **35**

53 Contender 51: programs over the iteration operators **36**

54 Contender 52: languages over the iteration operators, stacked **36**

55 Contender 53: reflection iterated along the tree **36**

56 Contender 54: reflection as a primitive **36**

57 Contender 55: number languages stacked along the tree **37**

58 Contender 56: tree and number languages stacked jointly **37**

59 Contender 57: stacking heights chosen by programs **37**

60 Contender 58: a tower of stack-reflecting languages **38**

61 Contender 59: the tower of stack-reflecting languages along a tree **38**

62 Contender 60: a hierarchy of tower operators **38**

63 Contender 61: programs over the tower operators **39**

64 Contender 62: the tower language stacked along a tree **39**

65 Contender 63: programs over the stacking operator of the tower language **39**

66 Contender 64: the stacking-operator step, towered along a tree **40**

67 Contender 65: a hierarchy of step operators **40**

68 Contender 66: programs over closure and towering **40**

69 Contender 67: a reflective tower of closure operators **41**

70 Contender 68: programs over the reflective tower's step **41**

71 Contender 69: the reflective tower over the step language **41**

72 Contender 70: the reflection cycle repeated **41**

73 An independent tower **42**

74 Roadmap **43**

75 Status **46**

1 The game

Scott Aaronson's *Who can name the bigger number?* asks for the largest number one can describe concisely. Cody Roux's repository name-the-biggest-number adds two caveats that turn the game into one about *proof*. First, numbers must be *constructive*: computable in principle, with no Busy Beavers. Second, each new contender must come with a *formal proof* that it beats the previous one. The original contest uses Coq. Its last entry, Contender 5 by Samuel Gruetter, is the largest value of a raw STLC+NatRec term (Gödel's System T) of syntax depth at most 42.

The Lean edition. This repository keeps the contest but switches to Lean 4 (version 4.34.0). The rules (`README.md`) are: closed computable `Nat` definitions; a theorem `contender_(N-1)_lt_contender_N`; the axioms `propext` and `Quot.sound` allowed and `Classical.choice` forbidden; at most 15 s to check the definition and 60 s to check the proof, from a clean build; and no laziness or shenanigans. The axiom policy is deliberate. `Classical.choice` is the only one of Lean's three axioms that conflicts with computability, so it is the one that must go.

2 How to compare unimaginably large numbers

All numbers here are far too large to evaluate. The formal theorems compare them directly, but informal *size* is best discussed with the *fast-growing hierarchy* f_α indexed by ordinals: $f_0(n) = n+1$,

$f_{\alpha+1}(n) = f_\alpha^n(n)$, and $f_\lambda(n) = f_{\lambda[n]}(n)$ at limits. A total language L whose programs are all provably terminating in a theory T has its bounded-size maxima growing roughly like $f_{|T|}$, where $|T|$ is the proof-theoretic ordinal of T. Two heuristics organize everything below.

1. **The base language dominates.** System T is calibrated by ε_0. T plus one type of countably branching trees, with recursion into all finite types, is calibrated by the Bachmann–Howard ordinal BH $= \psi(\varepsilon_{\Omega+1})$. With tree classes at every finite level the calibration is $\psi(\Omega_\omega)$, the ordinal of Π^1_1-CA$_0$.

2. **Reflection multiplies.** Relativizing a language L to an oracle that grows like f_β yields growth like $f_{\beta+|L|}$. Iterating this along an index of height γ yields $f_{|L|\cdot\gamma}$. Explicit ordinal notations without collapsing functions stay below BH, so a stack of reflections over a weak base cannot catch up with a stronger base.

These are informal calibrations, not theorems; the formal content of every contender is a strict inequality checked by Lean's kernel.

3 Contender 5, exactly

`BusyLean/Contender5.lean` ports `Contender.v` one definition at a time. The port keeps every quirk that can affect the value: raw ill-typed terms are interpreted with `error` defaults; variables use reversed de Bruijn indices; lambdas carry both annotations and cast their body; arrow casts recurse only when `type_eqb` succeeds; the enumeration order is the original one, including right-nested `++`; and the winner is chosen by `maxBy`'s strict update, starting from `t0`. A smoke test reproduces the reference table: $0, 2, 10, 146, 24976$ terms and maxima $0, 0, 1, 2, 3$ at depths 0–4.

Agreement of an independent port. Proposal P5 re-implemented the old language *inside* its larger W-language, with W-aware casts and a plain maximum instead of `maxBy`, and asserted without proof that the result is Contender 5. We prove it:

Theorem 1 (`PortAgreement.largest_eq_oldMaximum`)**.** *For every depth n,*

$$\texttt{C5.largestSTLCNatRecNatOfDepth}\ n \;=\; \texttt{BigNumber.oldMaximum}\ n.$$

The proof is a logical relation between the two interpreters: equality at $\mathbb{N}$, preservation at arrows. That avoids function extensionality. Casts in both directions preserve the relation, including the error branches. The fundamental lemma covers reversed lookup, binder casts and natural recursion. P5's enumeration is the translated image of the original, in the same order, and the `maxBy` winner attains the maximum.

4 Five proposals

Five proposals arrived as archives; `proposals/REVIEW.md` has the details. All were rebuilt on Lean 4.34.0, and the two that had never been compiled (P4, P5) needed small repairs, recorded in separate commits.

	base language	amplifier	informal size
P1	System T	FGH over explicit Veblen-style trees	$f_{\varepsilon_0+\alpha}$, $\alpha < \mathrm{BH}$
P2	T + Brouwer trees + oracles	forest reflection on explicit trees	$f_{\mathrm{BH}\cdot\alpha}$
P3	T + unary oracle	reflection on explicit trees	$f_{\varepsilon_0\cdot\alpha}$
P4	T + Brouwer trees + oracle	ramified reflection	$f_{\mathrm{BH}\cdot\alpha}$
P5	T + W-types at all finite types	two diagonals	$f_{\psi(\Omega_\omega)+\omega^2}$

P5 has by far the strongest base and the weakest amplifier, which suggests combining it with the others' amplifiers.

5 Contender 6

5.1 The object language

The language is P5's. Types are generated by $A, B ::= \mathbb{N} \mid A \to B \mid W\,A$, where $W\,A$ is the type of well-founded trees branching over A. Terms are raw (ill-typed terms default), with natural recursion and a W-recursor into every type. Because A ranges over all types, the language has tree classes of every finite level, $W\mathbb{N}$, $W(W\mathbb{N})$, ..., and its calibration is $|L| \approx \psi(\Omega_\omega)$.

5.2 Oracles for free

P5's interpreter resolves variables by oldest-first levels. Evaluating a program in the one-element environment $[\langle \mathbb{N} \to \mathbb{N} \to \mathbb{N}, fs\rangle]$ therefore makes `var 0` denote the oracle family fs everywhere, even under binders. No new syntax is needed, and the smoke test confirms it: `var0 3 4` evaluates to $fs(3,4)$, and so does `(fun y => var0 y y) 5` with the family applied at $(5,5)$.

5.3 Reflection along the language's own trees

```
def reflect (fs : Family) (n : ℕ) : ℕ :=
  n + 1 + max (prefixMax (fun i => fs i n) n)
              (maximum (evalWith fs) (allTerms n))
def stage (seed : Family) : WTree ℕ → ℕ → ℕ
  | .leaf   => reflect seed
  | .node f => reflect (fun i => stage seed (f i))
def treesOfDepth (k : ℕ) : WTree ℕ :=
  .node fun i => defTree (nthD (allTerms k) i .zero)
def index : WTree ℕ := .node treesOfDepth
def contender_6 : ℕ := stage p5Family index 42
```

At a node the oracle family *is* the family of child stages, so a program may call any child at a computed index (P2's forest reflection). The new ingredient is `index`: its k-th child joins every $W\mathbb{N}$ tree denoted by a program of depth $\le k$. The hierarchy therefore runs along the language's own ordinals, all of $|L|$, instead of an explicit notation system below BH. By the multiplication heuristic, Contender 6 is about $f_{|L|\cdot|L|}(42)$.

5.4 What is proved

- `contender_5_lt_contender_6`: the exact port of Contender 5 is below Contender 6.
- `p5_lt_contender_6` and `p5Family_lt_finalFamily`: P5 is beaten at every input.

- `defined_tree_eventually_lt`: every tree the language defines is eventually dominated.
- `program_lt_finalFamily`: every program of depth $\leq n$, run with the final oracles, is exceeded.

All of them depend only on `propext`. The definitions check in about 5 s and the proofs in about 6 s. A machine-checked audit shows that the definition never mentions `contender_5` or its winner selection. It also reports, candidly, that P5's seed contains the old language's maximum as a function of depth.

6 Engineering lessons

- **Toolchain drift changes axiom lists.** In Lean 4.34 a wildcard match arm compiles to a `_sparseCasesOn` helper built on `Nat.land`. `Nat.bitwise`'s termination proof uses `propext`, so "axiom-free on 4.19" did not survive the upgrade. The same happens to core `xs[i]?`. Exhaustive matches avoid it.
- **Tactics carry axioms.** `omega`'s certificates use `funext`, and so `Quot.sound`. Two calls were replaced by explicit lemma chains.
- **Never let the unifier normalize a contender.** Writing `iterate f 41 (f 42)` against `iterate f 42 42` made Lean try to evaluate the number. Generalizing the statement first, so that only $42 = 41 + 1$ is checked, fixes it.
- **Prove inequalities by inclusion, not monotonicity.** Reflection is not monotone in its oracle, because a program may test oracle values. Every comparison therefore exhibits an explicit occurrence: a visible oracle index or an enumerated program.

7 Contender 7: impredicative polymorphism

Contender 6 lives below the next collapsing level: everything it does is predicative. The next qualitative jump is *impredicative polymorphism*. The numeric functions definable in Girard's System F on Church numerals are exactly those provably total in second-order arithmetic Z_2, and $\psi(\Omega_\omega)$ is merely the ordinal of Π^1_1-CA_0, a small fragment of Z_2.

7.1 Why Lean can host System F

System F has no set-theoretic model in Lean's predicative `Type`: a model would have to interpret $\forall X.A$ as a product over *all* types, including itself. Girard's normalization proof needs much less. It interprets types as *reducibility candidates*, predicates on terms, and quantifies over all candidates. In Lean a candidate is a structure whose main field is $P : \mathrm{Tm} \to \mathrm{Prop}$. A statement "$\forall C : \mathrm{Cand}, \ldots$" is again a proposition, because `Prop` is impredicative. The normalization proof therefore goes through, even though there is no semantic model.

7.2 The formalization

1. **Untyped core** (`SystemF/Syntax`, `Reduction`): de Bruijn terms with an oracle constant `orc`; parallel substitution with its composition algebra; full β-reduction plus the δ-rule $\texttt{orc}\,\overline{n} \to \overline{f(n)}$ on canonical Church numerals; SN as accessibility.

2. **Types** (`Types`): de Bruijn System F types and Curry-style typing of *erased* terms. A flag enables the oracle rule; numerals are typed without it.

3. **Candidates** (`Candidates`): CR1–CR3, where *neutral* means neither a λ nor the oracle (`orc a` can reduce at the head); the arrow construction; the intersection $\forall$; the interpretation $[\![A]\!]_\rho$; its renaming and substitution lemmas, proved as *equalities* of candidates via funext, propext and proof irrelevance; β-closure for λ; and adequacy. The oracle is justified afterwards. Numerals lie in every interpretation of `nat` (adequacy without the oracle rule), so the δ-reduct of $\texttt{orc}\,a$ stays inside the candidate.

Theorem 2 (`F.sn_of_typed`)**.** *Every term typable in System F with the oracle is strongly normalizing under $\beta\delta$, for every oracle function f. The proof uses only* ***`propext`*** *and* ***`Quot.sound`****.*

7.3 From programs to numbers

Programs are Church-style terms (annotated λ, explicit type abstraction and application), so type inference is decidable. `infer_sound` states that inference implies the Curry typing of the erased term. A program's value is computed as follows: check that it is closed of type $\texttt{nat} = \forall X.(X \to X) \to X \to X$; erase it; normalize it by the normal-order step function `nstep`, iterated by well-founded recursion on the SN certificate (the compiler runs this as an ordinary loop); and read the Church numeral. A smoke test shows the pipeline computing: $2^3 \mapsto 8$ and $3^{2^2} \mapsto 81$ by polymorphic instantiation, oracle calls $\texttt{orc}\,\overline{4} \mapsto 40$ for $f(n) = 10n$, and 0 for ill-typed programs.

7.4 The contender

```
def reflectF (f : ℕ → ℕ) (n : ℕ) : ℕ :=
  n + 1 + max (f n) (maximum (fvalue f) (ctmsUpTo n))
def F7 (n : ℕ) : ℕ := reflectF finalFamily n
def contender_7 : ℕ := F7 (F7 42)
```

The oracle is Contender 6's family. System F programs can call it and amplify it with second-order strength. The outer `F7` runs System F at a syntax budget already larger than Contender 6. Proved: `contender_6_lt_contender_7`, `finalFamily_lt_F7` (at every input), and `fvalue_lt_contender_7` (every System F program of depth $\leq$ `F7 42` is exceeded). The audit confirms that the constant `contender_6` itself never occurs in the definition; only Contender 6's family occurs, as the oracle.

Size. By the heuristics of Section 2, Contender 7 is about $f_{|\mathsf{Z}_2|}$ applied at an input beyond Contender 6. Here $|\mathsf{Z}_2|$ is the proof-theoretic ordinal of second-order arithmetic. Nobody has computed it, but it lies far beyond every ordinal named in this article.

8 Contender 8: type operators

System F_ω adds *type operators*. Kinds are $\kappa ::= \star \mid \kappa \to \kappa$; type constructors are simply typed λ-terms over kinds, with $\to$ and $\forall\kappa$ as constants of kind $\star$. Programs may now abstract over type operators. A typical example is $\Lambda F{:}\star \to \star.\, \lambda x{:}F\,\texttt{nat}.\, x$. The numeric functions definable in F_ω are those provably total in higher-order arithmetic, beyond second-order arithmetic.

8.1 Candidates over kinds

The semantic domain is defined by recursion on kinds: $\mathrm{Sem}(\star) = \mathrm{Cand}$ and $\mathrm{Sem}(\kappa_1 \to \kappa_2) = \mathrm{Sem}(\kappa_1) \to \mathrm{Sem}(\kappa_2)$, with the arrow on the right being an ordinary Lean function type. A type operator therefore *denotes a function on candidates*, and type-level β-reduction is literally function application. The interpretation recurses on syntax at an *expected kind*, returning a default element on kind mismatch. This is the same dynamic-cast idea as the raw interpreters of Contenders 5 and 6, and it keeps the definition structural. Type application records its argument's kind for this purpose. The substitution lemma, the instantiation lemma and the invariance of the interpretation under β-conversion are proved for well-kinded types. The beta case of invariance is one line: function application plus the instantiation lemma.

Theorem 3 (`Fw.sn_of_typed`)**.** *Every term typable in System F_ω with the oracle is strongly normalizing, using only* ***`propext`*** *and* ***`Quot.sound`****.*

8.2 Deciding type equality, soundly

Church-style checking must compare types up to β. Instead of proving a second normalization theorem for the type level, the checker compares *fuel-bounded* normal forms. Every normal-order step is a genuine β-step, so equal results witness joinability, and `Fw.infer_sound` inserts the corresponding conversions. A program needing more fuel is merely rejected. That loses programs but never admits an ill-typed one, and Contender 8 uses the syntax budget as the fuel. The smoke test checks $(\lambda x{:}(\lambda X{:}\star.X)\,\mathtt{nat}.\,x)\,\overline{3} \mapsto 3$ (rejected with fuel 0), and quantification over an operator, $(\Lambda F{:}\star\to\star.\lambda x{:}F\,\mathtt{nat}.x)\,[\lambda X.X]\,\overline{5} \mapsto 5$.

8.3 The contender

```
def family7 (n : ℕ) : ℕ := F7 (F7 n)       -- family7 42 = contender_7
def F8 (n : ℕ) : ℕ := reflectFw family7 n
def contender_8 : ℕ := F8 (F8 42)
```

A first attempt used `F7` itself as the oracle and tried to compare `F7 (F7 42)` with `F8 (F8 42)`. That needs monotonicity of reflection, which fails. Taking Contender 7's whole construction as the family makes the comparison immediate: `family7 42 < F8 42 < F8 (F8 42)`.

9 Contender 9: reflecting F_ω along the W-language's trees

9.1 Why not the Calculus of Constructions

The natural next rung, the Calculus of Constructions, turns out *not* to be a growth-rate jump. In pure CoC, term dependencies can be erased into F_ω (Geuvers and Nederhof), so the numeric functions CoC represents on Church numerals are essentially those of F_ω. This is why Loader's number sits in the F_ω/CoC class, and Contender 8 already lives there. A genuine jump beyond higher-order arithmetic requires *runtime-to-type dependency with large elimination*: types computed by recursion on input numbers, so that a single program uses types of unbounded order. That is the realm of the Calculus of Inductive Constructions, whose normalization needs a realizability model with universes. It is a much larger project, described in the roadmap.

9.2 A multiplicative step

While that project is pending, Contender 6's amplifier applies to any total language with an oracle. Contender 9 reflects F_ω along Contender 6's self-referential index, the join of every tree the W-language defines:

```
def stage9 (seed : Family) : WTree ℕ → ℕ → ℕ
  | .leaf   => reflect9 seed
  | .node g => reflect9 (fun i => stage9 seed (g i))
def F9 (n : ℕ) : ℕ := stage9 (fun _ => family8) index n
def contender_9 : ℕ := F9 (F9 42)      -- family8 n = F8 (F8 n)
```

`reflect9 fs n` exceeds the visible oracle values $fs(i, n)$, $i \le n$, and every F_ω program of depth $\le n$. F_ω's oracle is unary, so the family is packed as $\texttt{orc}\,(2^i(2m+1)) = fs(i, m)$. Programs can compute these codes and so call any child at any input; the theorem `unpair2_code` shows that the decoder inverts the code. The index trees must still come from the W-language: F_ω has no set-theoretic model, so its programs cannot hand trees back to Lean. By the multiplication heuristic, Contender 9 is about $f_{|\mathsf{Z}_\omega|\cdot\psi(\Omega_\omega)^2}$. Proved: `contender_8_lt_contender_9`, `family8_lt_F9` (at every input) and `defined_tree_eventually_lt9`.

9.3 Budgets for a growing stack

Each contender seeds itself with its predecessor's family, so its definition transitively imports the whole history. The full chain took 16–30 s under parallel load on the reference laptop. A fixed budget for the whole stack would eventually be exhausted by history alone. The rules now treat earlier accepted contenders like the precompiled core library: the 15 s/60 s budgets apply to the new contender. Contender 9's definition checks in about 2 s and its proofs in about 3 s, and the full clean build of all contenders takes about 28 s.

10 Contender 10: kind polymorphism

10.1 Unbounded orders without dependent types

Each F_ω program uses finitely many kinds, fixed in its text, and is provably total in higher-order arithmetic of some finite order. To get past that, a single program must reach kinds of *unbounded* order, chosen at run time. Full dependent types with large elimination would do it. A much cheaper mechanism suffices: types that quantify over *kinds*, $\forall\kappa.A$ (of kind $\star$). A step function

$$g : \ \forall\kappa.\ B(\kappa) \to B(\kappa \to \star),$$

iterated m times through the impredicatively encoded existential $\exists\kappa.B(\kappa) = \forall Y\colon\star\,.\,(\forall\kappa.\,B(\kappa) \to Y) \to Y$, climbs m orders, where m is a run-time number. A totality proof for such programs needs reducibility candidates for all finite types uniformly, which is beyond higher-order arithmetic.

Consistency. In pure type system terms this adds the rule $(\triangle, \star)$, types quantifying over the sort of kinds, but not $(\triangle, \square)$, kinds quantifying over kinds. The latter would give Girard's System U^-, which is inconsistent by Hurkens' paradox. With only $(\triangle, \star)$, kinds have no binders, so kind variables always range over closed kinds, a countable set. The Lean proof below is semantic, so it certifies normalization, and with it consistency.

10.2 The model

A *kind valuation* θ maps kind variables to closed F_ω kinds, and kind annotations are evaluated under θ. The single new semantic ingredient is

$$[\![\forall\kappa.A]\!]_{\theta,\rho} = \bigcap_{K \text{ closed}} [\![A]\!]_{K\cdot\theta,\rho},$$

a countable intersection of candidates. The renaming and substitution lemmas for kind variables hold unconditionally. Adequacy gains two cases: kind generalization (a fresh kind extends θ, and contexts are kind-shifted) and kind instantiation (choose the evaluated kind).

Theorem 4 (`Fk.sn_of_typed`)**.** *Every term typable in kind-polymorphic System F_ω with the oracle is strongly normalizing, using only* `propext` *and* `Quot.sound`*.*

The Church-style checker extends Contender 8's with kind abstraction and application, again proved sound. A smoke test types $\Lambda\kappa.\Lambda F{:}\kappa.\lambda x{:}\mathtt{nat}.x$ as $\forall\kappa.\forall F{:}\kappa.\,\mathtt{nat} \to \mathtt{nat}$. It runs it at kind $\star$ (giving 4) and at kind $\star \to \star$ with the identity operator (giving 6), and rejects an ill-kinded instantiation.

10.3 The contender

```
def family9 (n : ℕ) : ℕ := F9 (F9 n)      -- family9 42 = contender_9
def F10 (n : ℕ) : ℕ := reflectFk family9 n
def contender_10 : ℕ := F10 (F10 42)
```

Proved: `contender_9_lt_contender_10`, `family9_lt_F10` (at every input), and `fvalue_lt_contender_10`. The new definition modules, four for the language plus the contender, check in about 12 s.

11 Contender 11: reflecting the kind-polymorphic language

Contender 11 applies Contender 9's amplifier to Contender 10's language: forest reflection of kind-polymorphic F_ω along Contender 6's index, with oracle families packed into the unary oracle as before.

```
def F11 (n : ℕ) : ℕ := stage11 (fun _ => family10) index n
def contender_11 : ℕ := F11 (F11 42)      -- family10 n = F10 (F10 n)
```

As with Contender 9, the gain is multiplicative, of about $|\mathrm{F}^\kappa_\omega| \cdot \psi(\Omega_\omega)^2$, where F^κ_ω denotes Contender 10's kind-polymorphic language.

12 A second batch of proposals, and Contender 12

12.1 Five more proposals

Five more archives arrived, all proposing a "Contender 7" on top of this repository's Contender 6, written before Contenders 7–11 existed. All five definitions compile under Lean 4.34. Two proposals compile completely: P6 (operator reflection), whose author had not compiled it, and P7 (critical-point reflection). The other three (P8 ramified worlds, P9 model ramification, P10 semantic mirror) fail in their proofs.

All five amplify the W-language, of strength about $\psi(\Omega_\omega)$. P6 lifts reflection to *transformers* of families and iterates the lift, which exponentiates the index height. P7 stacks explicit Veblen-style critical-point notations above the language-defined index. P8–P10 iterate world or model constructions in which a lower world's whole hierarchy is exposed to the next world's programs. The resulting ordinals range from $|L|^{|L|}$ to Γ-type closures above $|L|$, all far below second-order arithmetic. Informally, the whole batch is vastly smaller than Contender 7 (System F), let alone Contenders 8–11.

12.2 Reusing the generic ideas

Two of the ideas are generic and compose with stronger languages. P6's operator stage needs only that its base transformer be *progressive*: every visible seed value lies below every output. P7's notations apply above any Lean tree. Contender 12 adopts both, with attribution in the source, over the kind-polymorphic F_ω reflection:

```
def fkTransformer : Transformer := fun fs _ n => reflect11 fs n
def tree12 := realize (hyper 42 (encode index))   -- P7 notations
def F12 (n : ℕ) : ℕ :=
  operatorStage fkTransformer tree12 (fun _ => family11) n n   -- P6
def contender_12 : ℕ := F12 (F12 42)
```

The base is progressive by one line (`oracle_lt_reflect11`), and P6's theorem `operatorStage_progressive` then gives `family11_lt_F12` at every input, with no structural facts about `tree12` required. The gain over Contender 11 is again inside the language's ordinal: the index factor grows from $\psi(\Omega_\omega)^2$ to roughly $|L|^{\Gamma\text{-closure above } |L|}$. The world and model constructions of P8–P10 need an object language that manipulates trees as data, which the impredicative languages lack. They are recorded for a future tree-capable language.

13 Contender 13: a predicative universe of operators

13.1 The idea

Kind polymorphism (Contender 10) lets one program reach every *finite* order uniformly. The next step is an object that collects all finite orders at once. Letting kinds quantify over kinds would do it impredicatively, but that is Girard's System U^-, which is inconsistent. The predicative alternative is a new kind `dyn` whose semantic domain is the dependent sum

$$\mathrm{Sem}(\texttt{dyn}) \;=\; \textstyle\sum_{k\ \text{dyn-free}} \mathrm{Sem}(k),$$

which is exactly the type `SVal` of Contender 8's model. It is a universe of all finite-order operators that does not contain itself. Two type-level constructs reach it: $\texttt{pack}\,K\,A : \texttt{dyn}$ for $A : K$, and $\texttt{unpack}\,K\,X : K$ for $X : \texttt{dyn}$, with the conversion $\texttt{unpack}\,K\,(\texttt{pack}\,K\,A) \to A$. Both are restricted to *dyn-free* kinds, a syntactic condition.

Kind variables now come in two sorts. Level-0 variables range over dyn-free kinds and can appear in `pack`/`unpack`. Level-1 variables range over *all* kinds, including `dyn`, $\texttt{dyn} \to \star$, and so on. The existential climbing trick of Contender 10, applied to level-1 variables, therefore reaches orders $\omega + n$ for run-time n.

13.2 The model

Semantic kinds are closed syntactic kinds, which have decidable equality, so the dynamic read `SVal.get` of Contender 6 still works. A decoding `toFw` sends dyn-free closed kinds to F_ω kinds, with $\mathrm{Sem}(K) = \mathrm{Sem}_{F_\omega}(k)$ whenever $\texttt{toFw}\,K = \texttt{some}\,k$. `pack` is `SVal.mk` through that cast, and `unpack` is `SVal.get` through the reverse cast. The lemma `unpack_pack` states that the read-back of a packed value is the value itself, which is what makes the new conversion sound. Level-0 kind variables are evaluated to F_ω kinds and level-1 variables to arbitrary kinds; $\forall\kappa_0$ and $\forall\kappa_1$ are intersections over the respective ranges.

Theorem 5 (`Fd.sn_of_typed`)**.** *Every term typable in dynamic-kind System F_ω with the oracle is strongly normalizing, using only* `propext` *and* `Quot.sound`*.*

The smoke test exercises all of it. A program typable only through $\texttt{unpack}\star(\texttt{pack}\star\texttt{nat}) \equiv \texttt{nat}$ evaluates to 3. Level-1 polymorphism instantiated at `dyn` evaluates to 5. Instantiating a level-0 variable with `dyn`, and packing at a kind containing `dyn`, are both rejected.

13.3 The contender

```
def family12 (n : ℕ) : ℕ := F12 (F12 n)    -- family12 42 = contender_12
def F13 (n : ℕ) : ℕ := reflectFd family12 n
def contender_13 : ℕ := F13 (F13 42)
```

Proved: `contender_12_lt_contender_13`, `family12_lt_F13` at every input, and `fvalue_lt_contender_13`. The new definition modules check in about 7.5 s.

13.4 A third batch of proposals

Six more proposals (P11–P16) arrived, written against Contender 10 (and one against Contender 6). Three compile completely (P11, P14, P15). The other three fail in their proofs, all through the same trap: a proof step that makes Lean compare or unfold a term containing a contender-sized number. P13's proof module did not terminate within minutes. Their amplifiers (self-fed iteration counts, crowns above the index, finite diagonal ranks of the autonomous lift) each add about a factor of ω to the index ordinal. Contender 12's tree-indexed operator stage subsumes them, and Contender 13's language exceeds them. P15 independently rediscovered Contender 11's exact construction.

14 Contender 14: the ladder of universes

14.1 The idea

`dyn` collects the operators of every dyn-free kind, but it never contains itself, and nothing collects operators whose kinds mention `dyn`. The predicative answer is a ladder. The level-ℓ language has atomic kinds $\texttt{dyn}_0, \ldots, \texttt{dyn}_{\ell-1}$, and atom $\texttt{dyn}_i$ denotes the `dyn` of the level-i language. Two type-level constructs move operators into and out of an atom:

$$\frac{A : K \qquad K \text{ fits } a}{\texttt{apack}\,a\,K\,A : a} \qquad \frac{X : a \qquad K \text{ fits } a}{\texttt{aunpack}\,a\,K\,X : K} \qquad \texttt{aunpack}\,a\,K\,(\texttt{apack}\,a\,K\,A) \;\longrightarrow\; A,$$

where K *fits* a when K is built from $\star$, $\to$ and atoms strictly older than a. The top-level `dyn` and both sorts of kind variables stay as in Contender 13. So a program can pack an operator of kind $\texttt{dyn}_0 \to \star$ into $\texttt{dyn}_1$ and read it back.

14.2 One normalization proof for every level

Writing a new model for each level would never finish, so the language and its model are *parametric*. The syntax (`SystemFL`) is parametric in a type α of atoms with decidable equality and an order (`AtomOrder`). The model takes a `Universe`: a domain $\mathrm{El}(a)$ for each atom, a packing map from the base kinds that fit a, a reading map, and the law

$$\mathrm{rd}_a\, k\, (\mathrm{pk}_a\, k\, v) = v.$$

Base kinds are $\star$, $\to$ and atoms, interpreted with $\mathrm{Sem}(a) = \mathrm{El}(a)$. `dyn` is the sum over base kinds, and `apack`/`aunpack` are pk/rd through the decoding cast. The read-back law is exactly what the new conversion needs.

Theorem 6 (`FL.sn_of_typed`)**.** *For every universe U, every term typable in the ladder language over U's atoms (with the oracle) is strongly normalizing.*

The ladder itself (`SystemFL/Ladder.lean`) instantiates this theorem at every level:

- atoms are $\texttt{Atom}\,0 = \texttt{Empty}$ and $\texttt{Atom}\,(\ell+1) = \texttt{Option}\,(\texttt{Atom}\,\ell)$. The new atom `none` lies above all older atoms $\texttt{some}\,a$;
- `nextUniv` interprets the new atom as `BVal` of the previous universe, i.e. as its `dyn`, and keeps the older atoms;
- packing and reading go through `lower`, which recognizes the level-$(\ell+1)$ base kinds that avoid the new atom and turns them into level-ℓ base kinds. Semantic domains agree along it, so the read-back law at level $\ell+1$ reduces to the law at level ℓ.

One engineering point matters. pk receives a proof that the kind fits, so the lowered kind is extracted with `Option.get` rather than a dependent `match`. The law then needs no rewriting under a dependent match, the usual source of ill-typed motives.

14.3 The contender

```
def ladderUpTo (n : ℕ) := ctmsUpTo (Atom.all n) n     -- level-n programs
def reflectFL (f : ℕ → ℕ) (n : ℕ) : ℕ :=
  n + 1 + maxN (f n) (maximum (fvalue n f n) (ladderUpTo n))
def family13 (n : ℕ) : ℕ := F13 (F13 n)    -- family13 42 = contender_13
def F14 (n : ℕ) : ℕ := reflectFL family13 n
def contender_14 : ℕ := F14 (F14 42)
```

The value of a level-ℓ program is computed by the certified normalizer with the SN proof `FL.sn_of_typed` applied to the level-ℓ universe `Univ f`. The reflection diagonalizes *at the meta level*. Input n uses the level-n language, so no fixed rung keeps up with F_{14}. Proved: `contender_13_lt_contender_14`, `family13_lt_F14` at every input, and `fvalue_lt_contender_14`. The smoke test packs λ_:$\texttt{dyn}_0.\,\texttt{nat}$ into $\texttt{dyn}_1$ and applies its read-back to a packed `nat`, which evaluates to 4. It also confirms that an atom cannot hold its own kind.

The first budget run hit the 15 s definition limit (16.0 s on a loaded machine). Profiling showed the kind checker's soundness proof taking 2.4 s, with 6×6 case splits inside nested matches. Factoring the checker into small result functions (`arrK`, `appK`, `apackK`, ...) with standalone inversion lemmas cut that module from 5.8 s to 3.2 s. The definitions now check in about 10 s.

15 A fourth batch of proposals, and Contender 15

15.1 Four proposals over Contender 13

Four more proposals (P17–P20) arrived while Contender 14 was being built. All of them target Contender 13. All keep its language, dynamic-kind F_ω, and make its reflection *family-aware*: programs receive a packed countable oracle family, and a prefix maximum over the visible coordinates makes the transformer progressive without any monotonicity. Three compile as submitted (P17, P19, and P20, whose package says Lean was never run). P18 fails at a single proof step. A `simpa` misses the definitional equality `operatorStage` b `leaf` $=$ `liftTransformer` b, and replacing it with `exact` repairs it. All audits pass with `propext` and `Quot.sound`.

Writing τ for the autonomous schedule above `tree12`, P18 and P19 stack operator stages into towers of height `family13` n or $n+1$, about $|F_d| \cdot \tau^\omega$. P17 and P20 independently found something stronger, a *rank hierarchy of transformer lifts*:

$$\mathrm{lift}_0 = \texttt{liftTransformer}, \qquad \mathrm{lift}_{k+1}(b) = \mathrm{stage}_{\mathrm{lift}_k}(\mathrm{lift}_k(b),\ \sigma),$$

where $\mathrm{stage}_L(b, \sigma)$ traverses the schedule σ, applying the lift L at every node. Rank $k+1$ uses the whole rank-k operator at every node, so each rank roughly exponentiates the index. Diagonalized at rank `family13` $n+1$, this gives about $|F_d| \cdot \varepsilon(\tau)$. It strictly generalizes Contender 12's single operator stage. All four exceed Contender 13 (each proves it), but by the base-language heuristic all four lie below the ladder of Contender 14.

15.2 Contender 15

The two routes compose. The lift hierarchy is generic over transformers, so it can be applied to the ladder's reflection instead of F_d's. Contender 15 adopts P17's modules unchanged, with attribution, and defines:

```
def reflectFLFamily (fs : Family) (n : ℕ) : ℕ :=
  n + 1 + maxN (prefixMax (fun i => fs i n) n)
    (maximum (FL.fvalue n (packFamily fs) n) (FL.ladderUpTo n))
def family14 (n : ℕ) : ℕ := F14 (F14 n)      -- family14 42 = contender_14
def schedule15 (n : ℕ) := realize (hyper (family14 n + 42) (encode tree12))
def F15 (n : ℕ) : ℕ :=
  diagonalLift (schedule15 n) (family14 n + 1) flFamilyTransformer
    (fun _ => family14) n n
def contender_15 : ℕ := F15 (F15 42)
```

Every proof follows an explicit occurrence: of the seed in the prefix maximum, of a child at coordinate 0 of a splice, or of rank k inside rank $k+1$. Proved: `contender_14_lt_contender_15`, `family14_lt_F15` at every input, and `fvalue_lt_F15` (every level-n ladder program of depth $\le n$ is exceeded). The definitions check in 2.9 s and the proofs in 3.1 s. Informally the size is about $|\mathrm{FL}_n| \cdot \varepsilon(\tau)$, diagonalized over the ladder level n.

16 Contender 16: a transfinite ladder

16.1 The length of the ladder is the lever

Contender 14 diagonalizes over finitely many universes. Letting a program choose its level at run time would add only about one more factor of ω. A much larger lever is the *length* of the ladder.

The ladder language was parametric in its atoms from the start, and only the `Option` tower of Contender 14 was finite. So the question becomes: for which atom orders can Lean build a universe?

Theorem 7 (`FL.wfUniv`)**.** *For every type of atoms whose decidable order is well-founded, there is a universe. Hence (`FL.sn_of_typed`) the ladder language over those atoms is strongly normalizing.*

The domain of atom a is defined by well-founded recursion (`WellFounded.fix`):

$$\mathrm{El}(a) \;=\; \textstyle\sum_{k \text{ fits } a} \mathrm{Sem}(k), \qquad \mathrm{Sem}(b) = \mathrm{El}(b) \text{ for atoms } b < a.$$

A value stores its kind together with a proof that the kind fits. Reading compares kinds, which is decidable, so the law $\mathrm{rd}(\mathrm{pk}\, v) = v$ holds. The unfolding equation `WellFounded.fix_eq` is not definitional, so it is used through casts. Because types are erased, `WellFounded.fix` at `Type` compiles without a `noncomputable` marker. The universe at a limit atom such as ω holds operators from *every* lower level at once.

16.2 Well-founded Cantor normal forms

A non-increasing list $[a_1, \ldots, a_k]$ denotes $\omega^{a_1} + \cdots + \omega^{a_k}$, and the lexicographic order on such lists is the ordinal order. `Ordinal.wf_sorted` proves this order well-founded whenever the exponents are accessible. The proof uses neither transitivity nor trichotomy; asymmetry comes from accessibility. A sorted list headed by h splits into a run h^{k+1} and a tail below h. Two such lists compare like the pairs (k, tail), which gives an induction on the head, then on the run length, then on the tail. With natural-number exponents this gives notations below ω^ω (`OmegaPow`). Applied hereditarily it will give ε_0.

16.3 The contender

```
def transfiniteUpTo (n : ℕ) := ctmsUpTo (omegaPowUpTo n) n   -- atoms: ≤ n terms, exponents < n
def family15 (n : ℕ) : ℕ := F15 (F15 n)
def F16 (n : ℕ) : ℕ :=
  diagonalLift (schedule16 n) (family15 n + 1) tTransformer (fun _ => family15) n n
def contender_16 : ℕ := F16 (F16 42)
```

`tTransformer` is family-aware reflection over the transfinite ladder, normalized by `sn_of_typed (wfUniv f omegaPow_wf)`. At depth n, programs reach levels up to $\omega^{n-1} \cdot n$. Proved: `contender_15_lt_contender_16`, `family15_lt_F16` and `tvalue_lt_F16`. The smoke test packs an operator of kind $\mathtt{dyn}_3 \to \star$ into the level-ω universe and evaluates it to 4. The opposite direction is rejected. The definitions check in 4.8 s and the proofs in 2.8 s.

17 Contender 17: a ladder of length ε_0

Notations below ε_0 are trees `node` $[a_1, \ldots, a_k]$ denoting $\omega^{a_1} + \cdots + \omega^{a_k}$. They are compared lexicographically, child by child, and a tree is normal when its children are normal and non-increasing. On arbitrary trees the lexicographic order is *not* well-founded. For example, $t_{k+1} =$ `node` $[0, t_k]$ descends forever starting from $t_1 = \omega$. So the sorted-list lemma cannot be applied to all trees.

The fix is to build normality into the Boolean element order, $\mathrm{lt}_P(a, b) := \mathrm{nf}(a) \wedge \mathrm{lt}(a, b)$, and apply the generic lemma *to that order*. Two short lemmas then finish the proof:

- `acc_of_children`: if the list of children of t is accessible (sorted-lexicographically), then t is accessible. A smaller normal tree has a smaller sorted list of children;

- `acc_nf`: every normal tree is accessible. By structural recursion its *head* child is accessible, so `acc_head` makes the list of children accessible. The other children lie below the head and need no recursion.

`Ordinal.epsilon0_wf` transfers this to the subtype of normal forms, and `FL.wfUniv` supplies the universes unchanged. At depth n, programs use atoms of height $\leq n$, i.e. levels up to towers $\omega^{\omega^{\cdots}}$ of height about n. The contender has the same shape as before, `F17 n := diagonalLift (schedule17 n) (family16 n + 1) eTransformer (fun _ => family16) n n`. Proved: `contender_16_lt_contender_17`, `family16_lt_F17` and `evalue_lt_F17`. The smoke test packs an operator over level ω into the level-ω^ω universe (value 5), and the opposite direction is rejected. The definitions check in 2.0 s and the proofs in 1.9 s.

18 Contender 18: a ladder of length Γ_0

18.1 Veblen terms and a case-split proof

Terms are sums `node` $[\varphi(a_1, b_1), \ldots, \varphi(a_k, b_k)]$ of principal terms (a mutual, nested inductive). Sums are compared lexicographically, and principal terms by Veblen's rule. The rule is implemented as a Boolean function by mutual well-founded recursion on sizes:

```
def ltP : P → P → Bool
  | .phi a b, .phi c d =>
      if ltV a c then ltVP b c d          -- b < phi(c,d)
      else if decide (a = c) then ltV b d
      else leVP a b d                     -- phi(a,b) <= d
```

A term is normal when every sum is non-increasing, hereditarily. No fixed-point condition is imposed, so non-canonical terms such as $\varphi(0, \varepsilon_0)$ are allowed. They simply sit somewhere in the order. The genuine Veblen normal forms are among the normal terms, so the order type is at least Γ_0.

Theorem 8 (`Veblen.acc_phi`, `gamma0_wf`)**.** *If a and b are accessible, so is $\varphi(a, b)$. Hence every normal term is accessible, and the order on normal terms is well-founded.*

The proof is Schütte's, reorganized so that it only follows the *definitional* case split of `ltP`. It inducts on a, then on b, then on the size of a predecessor $\varphi(e, f)$:

- $e < a$ and $f < \varphi(a, b)$. Then f is 0 or headed by a smaller principal term below $\varphi(a, b)$, accessible by the size induction, so `acc_head` makes f accessible. The induction on a applies to e;
- $e = a$ and $f < b$: by the induction on b;
- otherwise $\varphi(e, f) \leq b$: then the one-term sum $\varphi(e, f)$ is below or equal to b, hence accessible (`accP_of_le`).

Transitivity, trichotomy and ordinal arithmetic are never needed. Sums are handled by `SortedLex` as for ε_0.

One engineering note. The first version depended on `Classical.choice`. `omega` had been called on a goal of the form `AccP q`, after deriving an arithmetic contradiction. Because that goal is not decidable, `omega` used `Classical.byContradiction`. Writing `absurd hqs (by omega)` gives `omega` a decidable goal, and the axioms return to `propext` and `Quot.sound`.

18.2 The contender

`F18 n := diagonalLift (schedule18 n) (family17 n + 1) gTransformer (fun _ => family17) n n`, with `family17` $n = F_{17}(F_{17}\, n)$ and reflection over the ladder whose atoms are `Gamma0`. Proved: `contender_17_lt_contender_18`, `family17_lt_F18` and `gvalue_lt_F18`. The smoke test packs an operator over level ε_0 into the level-$\varphi(\varepsilon_0, 0)$ universe (value 6). The definitions check in 2.4 s and the proofs in 1.8 s.

19 Contender 19: a ladder of large-Veblen length

19.1 Klammersymbole

Binary Veblen generalizes to φ applied to an *argument array*: a finite list of entries (position, value), sorted by decreasing position. In Schütte's Klammersymbole the positions are themselves terms. Terms are now a three-way mutual inductive (sums, principal terms $\varphi(A)$, entries). Arrays are compared lexicographically by entries, with entries ordered by position and then value. This is exactly the order "first difference at the most significant position". Principal terms follow the generalized rule:

$$\varphi(A) < \varphi(B) \iff \begin{cases} A < B \text{ and every value of } A \text{ is } < \varphi(B), \text{ or} \\ A \neq B,\ A \not< B, \text{ and } \varphi(A) \leq \text{some value of } B. \end{cases}$$

Binary Veblen is the case of positions 0 and 1: $\varphi(1@1) = \varepsilon_0$ and $\varphi(1@2) = \Gamma_0$. With positions ranging over the notation itself, the normal forms reach the large Veblen ordinal.

19.2 Well-foundedness

The proof still follows only the comparison's case split. Two new ingredients are needed:

- `SortedLexQ`: the sorted-list lemma restricted to a Prop-valued predicate. An entry is *good* when it is normal and its position and value are accessible. Good entries are accessible (`accE`: induction on the position, then the value). Hence sorted arrays of good entries are accessible (`accQ_sorted`).
- `posAcc`: if $\Gamma < A$ as sorted arrays, every position of Γ lies below or at a position of A, so it is accessible when A's positions are.

`LVeblen.acc_phi` inducts on the array, then on the size of the predecessor $\varphi(\Gamma)$. In the first case the values of Γ are below $\varphi(A)$ and accessible by the size induction, and the positions by `posAcc`. So Γ is a smaller good array and the array induction applies. In the second case $\varphi(\Gamma)$ lies below a value of A. The whole module checks in about 2 s, and the axioms are `propext` and `Quot.sound`.

19.3 The contender

`F19 n := diagonalLift (schedule19 n) (family18 n + 1) lTransformer (fun _ => family18) n n`, with reflection over the ladder whose atoms are `LargeVeblen`. Proved: `contender_18_lt_contender_19`, `family18_lt_F19` and `lvalue_lt_F19`. The smoke test packs an operator over level ε_0 into the universe at the small Veblen ordinal $\varphi(1@\omega)$ (value 7). The definitions check in 4.3 s and the proofs in 1.8 s.

20 Contender 20: a ladder of Bachmann–Howard length

20.1 Collapsing, and why the case split no longer suffices

Buchholz-style terms with two levels are sums of principal terms $D_0 a$ ($\psi_0(a)$, countable) and $D_1 a$ ($\psi_1(a)$, at least Ω). Principal terms are compared lexicographically (level, then argument), and sums lexicographically, so every D_1 term lies above every D_0 term. This raw order is not well-founded:

$$D_1 0 \;>\; D_0 D_1 0 \;>\; D_0 D_0 D_1 0 \;>\; \cdots$$

Normal forms exclude this. $D_u a$ is normal only when every element of $G_u(a)$ is below a, where $G_u(D_v b) = \emptyset$ for $v < u$ and $\{b\} \cup G_u(b)$ otherwise. In the chain, $D_0 D_1 0$ is normal, but $D_0 D_0 D_1 0$ is not, because $D_1 0 \in G_0(D_0 D_1 0)$ is not below $D_0 D_1 0$. The normal forms reach $\psi_0(\varepsilon_{\Omega+1})$, the Bachmann–Howard ordinal.

20.2 The impredicative argument in Lean

Let M be the set of terms whose *top-level D_0-components* (reached through sums and D_1 only) are accessible.

1. `accM_V` (Claim 1): the order restricted to M is well-founded. This part is predicative. D_0 components are accessible by assumption, D_1 terms by induction on their argument, and sums by `SortedLexQ`.
2. `accD0` (Claim 2): if a is accessible within M, then $D_0 a$ is accessible. A predecessor $D_0 c$ has $c < a$. Its components $D_0 f$ satisfy $f < c$ by normality, hence $f < a$ by *transitivity*, and they are smaller, so an inner size induction puts c in M. The outer induction then applies to c.
3. Claim 3: every normal $D_0 c$ is accessible (size induction plus Claims 1 and 2). $D_1 b$ is accessible whenever b is, because its predecessors are D_0 terms or $D_1 c$ with $c < b$. Hence every normal term is accessible (`bachmannHoward_wf`).

This is the point where the well-foundedness proof itself becomes impredicative. Claim 1 is an accessibility statement *relative to* a set defined from accessibility, and Lean's inductive `Acc` carries it without any extra axiom. Transitivity, avoided until now, is needed in Claim 2. The order is purely lexicographic and structurally recursive, so transitivity is a short mutual induction. `SortedLexQ` gained `accQ_head_of_acc`, which starts from one accessible head rather than a global hypothesis. The module checks in about 2 s.

20.3 The contender

`F20 n := diagonalLift (schedule20 n) (family19 n + 1) bTransformer (fun _ => family19) n n`, with reflection over the ladder whose atoms are `BachmannHoward`. Proved: `contender_19_lt_contender_20`, `family19_lt_F20` and `bvalue_lt_F20`. The smoke test packs an operator over level $\psi_0(\Omega)$ into the universe at Ω (value 8). The definitions check in 2.9 s and the proofs in 1.9 s.

21 Contender 21: Buchholz ladders up to $\psi_0(\Omega_\omega)$

21.1 Why more levels are hard

With levels $0, 1, 2, \ldots$ the two-level argument of Contender 20 regresses. The order restricted to M_0 contains every $D_1 d$ with $d \in M_0$ arbitrary, and every $D_2 b$ lies above all of them. So Claim 1 at

level 0 needs a level-1 collapse *relative to* M_0, which needs a level-2 collapse relative to a smaller scope, and so on. There is no bound, because high levels hide inside low arguments: $\psi_0(\Omega_9) < \Omega_1$. The literature resolves this with Buchholz's distinguished sets.

21.2 A level bound turns it into a downward induction

For a fixed bound L on all levels, the regress stops. A *scope* is a predicate Z on principal terms, and the order is restricted to normal in-scope terms. For $u \le L+1$ let

> $R(u)$: if every in-scope term of level $< u$ is accessible and scopes are closed under arguments at levels $\ge u$, then every in-scope normal principal term is accessible.

$R(L+1)$ holds because every level is below $L+1$. `Buch.stepR` derives $R(u)$ from $R(u+1)$:

1. Restrict the scope to Z', the in-scope terms whose top-level u-components (through sums and higher levels) are accessible. $R(u+1)$ applied to Z' makes the Z'-order well-founded (Claim 1).
2. Induction along it, an inner size induction and transitivity make $D_u a$ accessible for every a accessible within Z' (Claim 2).
3. Every in-scope $D_u c$ is then accessible (Claim 3).
4. $R(u+1)$ applied to Z itself gives the conclusion.

$R(0)$ for the full scope gives `buchholz_wf L`: the notation with levels $\le L$ is well-founded. Its normal forms (for $D_u a$: $f < a$ for every top-level u-component $D_u f$ of a) include Buchholz's, so the order type is at least $\psi_0(\varepsilon_{\Omega_L+1})$. Relations are written as `LTQ (scope ∧ nf) ltP`, so `SortedLexQ` and `Subrelation.accessible` do all the bookkeeping between scopes.

21.3 The contender

As in Contender 14, the ladder is diagonalized at the meta level. Input n uses the atoms `Buchholz n`, with the universe `wfUniv f (buchholz_wf n)`. The ladder lengths therefore climb through $\psi_0(\varepsilon_{\Omega_n+1})$ towards Buchholz's ordinal $\psi_0(\Omega_\omega)$, the proof-theoretic ordinal of Π^1_1-CA$_0$. `F21 n := diagonalLift (schedule21 n) (family20 n + 1) zTransformer (fun _ => family20) n n`. Proved: `contender_20_lt_contender_21`, `family20_lt_F21` and `zvalue_lt_F21`. The smoke test (level bound 2) packs an operator over level $\psi_0(\Omega_2)$ into the universe at Ω_1 (value 9). The definitions check in 4.0 s and the proofs in 2.5 s.

22 Contender 22: a tower of relativized Buchholz notations

22.1 The barrier at $\psi_0(\Omega_\omega)$

Contender 21's argument needs a bound on the levels: the downward induction starts at $R(L+1)$. A single notation for all levels has no such start, and the relativized argument regresses forever. The literature crosses this barrier with Buchholz's *distinguished sets*: sets X such that, for each $\alpha \in X$, X agrees below the next regular above α with the well-founded part of a closure $C^\alpha(X)$. The maximal distinguished class is then shown to contain every notation. Arai's lecture notes present the technique for reflection notations. Porting it means formalizing a large part of the notation theory (closure sets, their monotonicity, and the comparison facts), so it remains on the roadmap.

22.2 Relativizing to a base

A cheaper step stays within finitely nested accessibility. The level-bounded systems are relativized to a type of *base atoms* with a transitive, well-founded order (`BuchholzBase.lean`). Atoms $B\,b$ lie below every collapse but may occur inside collapse arguments, so each collapse ranges over everything below it. The well-foundedness proof is Contender 21's `stepR`. Only the treatment of atoms changes: they are accessible in every scope by the base's own well-foundedness (`accB`), not by the level induction.

The union of all level-bounded systems over a base, ordered by the bound *first*, is again a transitive, well-founded, decidable order. It is well-founded by induction on the bound and then `acc_nf`. So it can serve as the base of the next round:

```
structure WFO where            -- order, transitivity, well-foundedness, enumeration
  T : Type
  ...
def tower : ℕ → WFO
  | 0 => WFO.empty
  | k + 1 => (tower k).next      -- all bounded systems over tower k
```

`tower 1` has order type $\psi_0(\Omega_\omega)$. Each further step collapses over everything below it, heuristically giving $\psi_0(\Omega_\omega \cdot k)$ for `tower k`.

22.3 The contender

Input n uses the ladder over `tower n`, with the universe `wfUniv f (tower_wf n)`. `F22 n := diagonalLift (schedule22 n) (family21 n + 1) twTransformer (fun _ => family21) n n`. Proved: `contender_21_lt_contender_22`, `family21_lt_F22` and `tvalue22_lt_F22`. The smoke test packs an operator over a base atom of `tower 2` into the universe at ψ_0 of that atom (value 10). The definitions check in 3.7 s and the proofs in 1.9 s.

23 Proposal P22 and Contender 23: transfinite ranks

Two more proposals arrived. P21 is a tower of operator stages over Contender 13's language. It compiles even though its package says it was never compiled, and it is subsumed by the rank hierarchy of lifts. P22 makes a real improvement to that hierarchy. The rank, a natural number in P17's version, becomes a *well-founded W-tree*:

```
def wDiagonalLift (schedule : WTree ℕ) : WTree ℕ → TransformerLift
  | .leaf => liftTransformer
  | .node f =>
      let lower := spliceLifts (fun i => wDiagonalLift schedule (f i))
      fun base => stageLift lower (lower base) schedule
```

At a node, the lifts of all children are spliced into one lift, which is used twice: to pre-amplify the base, and as the local lift along the schedule. The rank tree is an autonomous notation above the schedule, so the hierarchy has a large countable rank instead of ω. P22 compiles and passes the audit. It targets Contender 19's language, so it lies below Contenders 20–22. But the amplifier composes with any ladder.

Contender 23 adopts P22's modules unchanged, with attribution, and applies them to Contender 22's tower ladders:

```
def rank23 (n : ℕ) := realize (hyper (family22 n + 42) (encode (schedule23 n)))
def F23 (n : ℕ) : ℕ :=
  wDiagonalLift (schedule23 n) (rank23 n) twTransformer (fun _ => family22) n n
```

Proved: `contender_22_lt_contender_23`, `family22_lt_F23` and `tvalue22_lt_F23`. The definitions check in 2.6 s and the proofs in 3.9 s.

24 Contender 24: towers along well-founded indices

Contender 22's tower iterates "all level-bounded systems over the previous stage" along ℕ. Nothing forces the index to be ℕ. `TowerAlong.lean` builds the tower along any well-founded index J:

```
def towerAlong (J : WFO) (j : J.T) : WFO :=
  (sigma (sub J j) (fun i => towerAlong J i.1)).next   -- over all stages below j
termination_by j
def limitAlong (J : WFO) : WFO := sigma J (towerAlong J)
def roundTower : ℕ → WFO
  | 0 => tower 1
  | k + 1 => (roundTower k).limitAlong                  -- indexed by the last round
```

`sigma J F` is the union of a family of orders, ordered by index first. Its decidable equality, transitivity and well-foundedness (induction on the index, then on the element) are proved once and for all. The stage at j is heuristically of order type $\psi_0(\Omega_\omega \cdot (j+1))$. Since each round is indexed by the previous one, the rounds climb the fixed points of $x \mapsto \psi_0(\Omega_\omega \cdot x)$.

One implementation point. `WellFounded.fix` producing *data*, here records with order functions and enumerations, would need `noncomputable`, which the rules forbid. Recursion via `termination_by`, with a high-priority `WellFoundedRelation` instance built from J's well-foundedness, compiles normally. The smoke test computes 165 atoms of `roundTower 1` at depth 2 and runs a pack/unpack program between two of them.

Input n uses the ladder over `roundTower n`, with Contender 23's W-ranked lift hierarchy seeded with Contender 23's family. Proved: `contender_23_lt_contender_24`, `family23_lt_F24` and `rvalue_lt_F24`. The definitions check in 2.5 s and the proofs in 2.2 s.

25 Contender 25: well-foundedness for free

25.1 More proposals, and one decisive idea

Four more proposals arrived (P23–P26), all over Contender 19's language. P23 and P26 raise the lift hierarchy one type level (lifts of lifts). P24 rediscovers P22's W-tree-ranked hierarchy. They all compile, P23 after the same one-step `simpa` repair as P18, and none exceeds the current contenders. P25 contributes the decisive idea. Its atoms are *finite addresses in a computable W-tree*. Following an address is a computation, so validity is decidable. The order between addresses is well-founded *by structural recursion on the tree*, whatever ordinal the tree denotes.

All ladders since Contender 16 paid for their length with a hand-written well-foundedness proof, and that cost stopped the notations at Buchholz's ordinal. With addresses, the cost disappears. The ladder length is the rank of *any* tree that Lean can define, and Lean defines trees by structural recursion, with no proof obligations.

25.2 Addresses

`TreeAddress.lean` uses tree ordinals (zero, successor, countable limits, and suprema over two index types, used by collapsing) and the *strict descendant* order on valid addresses. P25 used immediate children. Accessibility of an address follows by recursion on the subtree it reaches (`acc_follow`). A strict descendant of a is a descendant of some child of a (`isDesc_split`), and addresses below a dead end are invalid. `addr_wf` uses only `propext`.

25.3 Collapsing functions by structural recursion

`TreeCollapse.lean` defines Buchholz-style collapsing on tree ordinals for all finite levels at once. Level-$(w{+}1)$ trees have suprema over all lower levels (lumped in one index type) and over level w. The argument space has suprema at every finite level. Then

```
def D : (v : ℕ) → TopT → LevT v
  | v, .zero => LevT.Omega v
  | v, .succ t => LevT.epsNext v (D v t)
  | v, .lim f => LevT.lim v fun n => D v (f n)
  | v, .supH g =>        -- below v: keep the supremum; for every j >= v: diagonalize
      ... D v (g (v + i, seq (v + i) n)) ...   -- seq j (n+1) = D j (g (j, seq j n))
```

Lean accepts this as structural recursion: every recursive call is on `g x` for some `x`. `D` depends on no axioms at all. The countable tree `bigTree n` collapses $n{+}1$ further ε-numbers above $\Omega_\omega + 1$, where $\Omega_\omega = \lim_k \Omega_k$. Heuristically its rank is $\psi_0(\varepsilon_{\Omega_\omega + n})$. That is beyond Buchholz's ordinal, and in exactly the range where a notation-based proof would need distinguished sets.

25.4 The contender

Input n uses the ladder over `Addr (bigTree n)`, with the universe `wfUniv f (addr_wf (bigTree n))` and the W-ranked lift hierarchy seeded with Contender 24's family. Proved: `contender_24_lt_contender_25`, `family24_lt_F25` and `avalue_lt_F25`. In the smoke test, an operator over the universe at address $(3, 5)$ packs into the root's universe (value 12). The definitions check in 3.5 s and the proofs in 2.2 s.

26 Contender 26: transfinitely many levels

Contender 25's collapsing functions have finitely many levels. With tree ordinals, the levels can themselves be countable tree ordinals (`TreeLevels.lean`). The type of everything below level a is defined by structural recursion on the level tree:

$$\mathrm{Low}(0) = \emptyset, \qquad \mathrm{Low}(a{+}1) = \mathrm{Low}(a) \oplus \mathrm{Lev}(a), \qquad \mathrm{Low}(\lim f) = \textstyle\sum_i \mathrm{Low}(f\,i) \oplus \mathrm{Lev}(f\,i),$$

with $\mathrm{Lev}(a)$ the trees with suprema over $\mathrm{Low}(a)$. Embedding a level into a higher one extends suprema by zero, so no retraction is needed.

A difficulty arises for arguments. Splitting a supremum into the parts below and above the target level would need a decidable order on levels, and tree orders are not decidable. The fix is to let suprema carry their level. The argument type has a constructor `supAt (c : CTree) (g : Low c → LTop)`, which is still strictly positive. Then

```
def DL : (a : CTree) → LTop → Lev a
  | a, .supAt c g =>
```

```
    match c, g with
    | .succ c', g => -- x0 = Omega c',  x(k+1) = DL c' (g x(k));  result lim_k DL a (g x(k))
    | .lim f, g   => -- the limit over i of the successor case at f i
    ...
```

Again every recursive call is on `g x`, so the definition is structural, and it uses only `propext`. `ladderTree n` iterates $\Lambda \mapsto \mathrm{DL}\,0\,(\varepsilon_{\Omega_\Lambda+1})$ $n{+}1$ times, starting from Contender 25's tree. Heuristically this climbs toward the fixed points of $\alpha \mapsto \psi_0(\Omega_\alpha)$. Input n uses the ladder over `Addr (ladderTree n)`, with the W-ranked lift hierarchy seeded with Contender 25's family. Proved: `contender_25_lt_contender_26`, `family25_lt_F26` and `lvalue26_lt_F26`. The definitions check in 4.4 s and the proofs in 4.9 s.

27 Contender 27: universes of level codes

In Contender 26 the levels are indexed by countable codes, so iterating $\Lambda \mapsto \psi_0(\Omega_\Lambda)$ stalls at a countable fixed point. To pass it, level codes must branch over *uncountable* level elements. Doing that in one step would be an inductive–recursive definition, which Lean does not have. The stratified version is available (`TreeUniverses.lean`):

```
structure Univ : Type 1 where
  Code : Type
  Low : Code → Type                          -- everything below level c
  sample : (c : Code) → ℕ → Option (Low c)   -- elements for diagonals
  top : Code
inductive NCode (U : Univ) : Type where      -- codes of the next universe
  | zero | succ (t : NCode U) | lim (f : ℕ → NCode U)
  | supAt (c : U.Code) (h : U.Low c → NCode U)
```

The next universe's `Low` is defined by structural recursion on `NCode U`, so its codes denote levels as large as the elements of the previous universe's levels. Iterating gives the tower `univ k`. Each universe has its own collapse `DN`, again by structural recursion. Successor codes diagonalize through the predecessor. Limit codes take limits. A code whose supremum ranges over an old `Low c` diagonalizes along sample elements of `Low c`. The samples are needed because these index types have no decidable equality, but the strength comes from the diagonal. `universeLadderTree n` collapses $\varepsilon_{\Omega_{\mathrm{top}}+1}$ in universe $n{+}1$ and uses no axioms at all. Heuristically the nesting $\Omega_{\Omega_{\dots}}$ deepens with n, toward the first fixed point of $\alpha \mapsto \Omega_\alpha$, an inaccessible-like level.

Input n uses the ladder over `Addr (universeLadderTree n)`, with the W-ranked lift hierarchy seeded with Contender 26's family. Proved: `contender_26_lt_contender_27`, `family26_lt_F27` and `uvalue_lt_F27`. The definitions check in 2.8 s and the proofs in 2.5 s.

28 Contender 28: universes along tree ordinals

Contender 27 stacks universes of level codes along $\mathbb{N}$. If a universe carries a *sequence* of large codes instead of one, the union of a sequence of universes is again a universe, with codes $\sum_i (F\,i)$.Code. Universes can then be stacked along any countable tree ordinal, by structural recursion (`TreeUniverses2.lean`):

```
def univT : CTree → Univ
  | .zero => Univ.base
  | .succ a => (univT a).next
  | .lim f => Univ.union fun i => univT (f i)
```

The number of stacked universes can be the previous round's collapsed tree: `nextRound` collapses inside the universe after `univT` of the previous tree, and `roundTree n` iterates this $n{+}1$ times from Contender 27's tree. No axioms are used. Input n uses the ladder over `Addr (roundTree n)`, with the W-ranked lift hierarchy seeded with Contender 27's family. Proved: `contender_27_lt_contender_28`, `family27_lt_F28` and `rtvalue_lt_F28`. The definitions check in 2.2 s and the proofs in 2.4 s.

29 Contender 29: higher-order universe operators

Contender 28 builds universes with two operations, *next* and *union*. Universe-building operators can themselves be iterated (`TreeUniverses3.lean`). For $F : \mathrm{Univ} \to \mathrm{Univ}$, let $\Phi\, F\, U$ be the union of the finite iterates $F^k(U)$, the closure point of F above U. Then $\Phi(\mathrm{next})$ jumps to the next universe closed under *next*, an inaccessible-like step. $\Phi(\Phi(\mathrm{next}))$ jumps to the next limit of those, and so on. Operators are indexed by tree ordinals by structural recursion:

$$\Phi_0 = \mathrm{next}, \qquad \Phi_{a+1} = \Phi(\Phi_a), \qquad \Phi_{\lim f}(U) = \bigcup_i \Phi_{f\,i}(U).$$

Each round collapses inside the universe after $\Phi_\Lambda(\mathrm{base})$, where Λ is the previous round's tree. `operatorTree n` runs $n{+}1$ rounds from Contender 28's tree and uses no axioms. Input n uses the ladder over its addresses, with the W-ranked lift hierarchy seeded with Contender 28's family. Proved: `contender_28_lt_contender_29`, `family28_lt_F29` and `otvalue_lt_F29`. The definitions check in 2.4 s and the proofs in 2.3 s.

30 Contender 30: closure functionals of all finite types

Contender 29 closes universe operators under iteration. The closure operation itself can be lifted to every finite type over universes (`TreeUniverses4.lean`):

$$A_0 = \mathrm{Univ}, \qquad A_{k+1} = A_k \to A_k, \qquad \mathrm{closure}_k\, f\, a = \bigcup_m f^m(a) \quad (f \in A_{k+1}),$$

with unions taken pointwise at every type. $\mathrm{closure}_0$ is Contender 29's Φ. $\mathrm{closure}_1$ closes Φ-like functionals under iteration, and so on. At input n the chain $\mathrm{closure}_n, \ldots, \mathrm{closure}_0$ is applied to Contender 29's operator Φ_Λ and then to the base universe. The resulting universe is collapsed to a countable tree, and the ladder runs over its addresses. No axioms are used. Heuristically each type level adds a further layer of closure points, a hierarchy of inaccessible-like universes of higher type. Proved: `contender_29_lt_contender_30`, `family29_lt_F30` and `tlvalue_lt_F30`. The definitions check in 2.1 s and the proofs in 2.0 s.

31 Contender 31: closure functionals of transfinite type

The finite types of Contender 30 extend along tree ordinals (`TreeUniverses5.lean`):

$$AT_0 = \mathrm{Univ}, \qquad AT_{a+1} = AT_a \to AT_a, \qquad AT_{\lim f} = \prod_i AT_{f\,i},$$

with pointwise unions everywhere and closure points at successor types. A canonical functional $\mathrm{step}_a \in AT_{a+1}$ is defined by structural recursion. It is the base operator at 0, closure at successors, and taken componentwise at limits. It is evaluated down to a universe along the canonical chain of lower elements, with unions at limit types. The type level at input n is Contender 30's tree, so the height of the type hierarchy is itself a collapsed tree. No axioms are used. Proved: `contender_30_lt_contender_31`, `family30_lt_F31` and `ttvalue_lt_F31`. The definitions check in 2.0 s and the proofs in 2.4 s.

Where this leaves the search. Contenders 25–31 all follow one pattern: a countable tree ordinal is defined by structural recursion, and the ladder runs over its addresses, with well-foundedness for free. Each step nests the previous construction inside a stronger recursion. The true sizes are heuristic; only the comparisons are proved. A qualitatively larger jump needs a stronger object language, for example dependent types with large elimination, which would require a new normalization proof.

32 Contender 32: a programming language for tree ordinals

Contenders 25–31 are each *one* combination of tree-ordinal operations. The natural meta-step is to make the combinations programs (`TreeLanguage.lean`). The language is Gödel's System T with a second base type of countable tree ordinals:

- types `nat`, `tree`, and arrows, each denoting a Lean type;
- numerals with a recursor; `tzero`, `tsucc`, `tlim`; a tree recursor that iterates along any tree at any type; and a primitive `boost : tree => tree`, which is Contender 31's construction as a function of its index tree.

Unlike the impredicative ladder language, this language needs no normalization proof. It is simply typed and predicative, so intrinsically typed terms have a direct *semantic* interpreter. Types denote Lean types, and `Tm.eval` maps terms to Lean values by structural recursion, using no axioms. A type checker turns raw terms into typed ones. `languageTree n` is the limit of Contender 31's tree and of all closed `tree` programs of size $\leq n{+}3$, so it contains transfinite and higher-type iterations of `boost`: Veblen- and Bachmann-style hierarchies over it, written as programs. The enumeration is hopeless to run beyond tiny sizes (3.5 million raw terms at size 3), but it is never run. The smoke test stays inside the first branch.

Proved: `contender_31_lt_contender_32`, `family31_lt_F32` and `lgvalue_lt_F32`. The definitions check in 2.7 s and the proofs in 2.2 s.

33 Contender 33: languages stacked on languages

Contender 32 takes all programs of bounded size over a primitive. The next step makes the whole language the next primitive (`TreeLanguage2.lean`):

$$\mathrm{langLift}(b)(t) = \lim_m \lim_i P_i(t),$$

where P_i ranges over the closed programs of type `tree => tree` and size $\leq m$ over the primitive b. So $\mathrm{langLift}(b)$ dominates every transfinite and higher-type iteration of b that the language can express. Stacking gives $\mathrm{boost}_{k+1} = \mathrm{langLift}(\mathrm{boost}_k)$ from Contender 31's construction. At input n the tree is boost_{n+1} applied to Contender 32's tree, and the ladder runs over its addresses. Proved: `contender_32_lt_contender_33`, `family32_lt_F33` and `skvalue_lt_F33`. The definitions check in 2.0 s and the proofs in 2.1 s.

34 Contender 34: languages stacked along tree ordinals

Contender 33 stacks the language $n{+}1$ times. The stacking height can itself be a tree ordinal (`TreeLanguage3.lean`):

$$\mathrm{boost}_0 = \mathrm{boost}_{31}, \qquad \mathrm{boost}_{a+1} = \mathrm{langLift}(\mathrm{boost}_a), \qquad \mathrm{boost}_{\lim f}(t) = \lim_i \mathrm{boost}_{f\,i}(t).$$

At input n, both the height and the argument are Contender 33's tree, and the ladder runs over the addresses of the result. Proved: `contender_33_lt_contender_34`, `family33_lt_F34` and `tsvalue_lt_F34`. The definitions check in 2.0 s and the proofs in 2.1 s.

35 Contender 35: impredicative trees

All the trees of Contenders 25–34 come from *predicative* constructions: collapsing functions, universes of codes, System T over trees, and stackings of that language. Contender 35 lets an impredicative language build the tree.

System FT. `SystemFT` is System F with a base type `tree` and three inert constants

$$\mathsf{con}_0 : \mathsf{tree}, \qquad \mathsf{con}_1 : \mathsf{tree} \to \mathsf{tree}, \qquad \mathsf{con}_2 : (\mathsf{nat} \to \mathsf{tree}) \to \mathsf{tree}.$$

There is no eliminator for `tree`. Programs compute with Church-encoded data and then build a tree with the constructors. A polymorphic Church tree of type $\forall X.\, X \to (X \to X) \to ((\mathsf{nat} \to X) \to X) \to X$ can be instantiated at $X = \mathsf{tree}$. So every tree that System F can describe becomes a real tree. The ordinals of such trees go beyond everything that predicative constructions reach.

Well-foundedness by candidates. The tree type is interpreted by an inductive candidate WT: a term is in WT when it is strongly normalizing, all its reducts are in WT, and $\mathsf{con}_1\, b$ has $b \in \mathrm{WT}$ while $\mathsf{con}_2\, g$ has $g\,\bar{k} \in \mathrm{WT}$ for every numeral $\bar{k}$. Condition CR3 of reducibility candidates is stated for *neutral* terms. Constructor-headed terms must not count as neutral, otherwise CR3 would force $\mathsf{con}_1\, b \in \mathrm{WT}$ without $b \in \mathrm{WT}$. With that change the adequacy proof of System F goes through, and `wt_of_typed` shows that every program of type `tree` is in WT. Because WT is inductive, the child relation (reduce, then take a constructor argument) is accessible. `toTree` reads the tree off the normal form by well-founded recursion on it. Like the normalizer it is compiled code, and it uses only `propext` and `Quot.sound`.

The contender. At input n, `ftTree n` is the limit of Contender 34's tree and the trees of all System FT programs of depth $\le n$, with Contender 34's family as the oracle. The ladder runs over its addresses and is amplified by the W-ranked lift seeded with family_{34}. Proved: `contender_34_lt_contender_35`, `family34_lt_F35` and `ftvalue_lt_F35`. The definitions check in 8.0 s, which now includes compiling System FT, and the proofs in 3.4 s.

36 Contender 36: ladders that build their own length

Contender 35 takes trees from System F. The ladder language over the addresses of a tree t is much stronger than System F, since it has a universe for every address. Contender 36 lets the ladder itself produce the next tree.

The ladder over FT terms. The kinds and types of the ladder language (`SystemFL/Types.lean`) do not mention terms. So `SystemFLT` reuses them and copies only typing, candidates, Church terms and the transfinite universes onto the terms of System FT. No typing rule for the constructors is needed. A program of the Church tree type

$$\mathsf{treeTy} = \forall X{:}\star\,.\; X \to (X \to X) \to ((\mathsf{nat} \to X) \to X) \to X$$

is applied, after erasure, to $\mathtt{con}_0, \mathtt{con}_1, \mathtt{con}_2$. Adequacy with X interpreted by the tree candidate gives `FLT.wt_of_treeTy`: the result is a well-founded tree, for every universe. This includes the universes `wfUniv` over the addresses of any tree.

Bootstrapping. One step turns a tree into a longer one:

$$\mathrm{ladderTree}(t) = \lim(t,\ \text{trees of all ladder programs of depth} \le n \text{ over } \mathrm{Addr}(t)).$$

`bootT` iterates the step along a tree index, with limits at limit nodes. At input n the index and the start are both Contender 35's tree: $\mathrm{bootTree}(n) = \mathrm{bootT}_{\mathrm{ftTree}(n)}(\mathrm{ftTree}(n))$. The final ladder runs over its addresses, amplified by the W-ranked lift seeded with family_{35}. Each ladder's programs thus define the length of the next ladder, so the construction approximates fixed points of the map from a tree to the trees definable over it.

Proved: `contender_35_lt_contender_36`, `family35_lt_F36` and `btvalue_lt_F36`. The definitions check in 9.7 s and the proofs in 3.0 s.

37 Contender 37: languages stacked over the ladder jump

Contender 36's bootstrap, read as an operator on trees, is a *jump*: $\mathrm{jump}_n(t) = \mathrm{bootT}_t(t)$ iterates "the trees of all ladder programs over $\mathrm{Addr}(t)$" along t itself. Contenders 33 and 34 stacked System T over trees on a fixed primitive. `stackOver` takes the primitive as a parameter:

$$\mathrm{S}^b_0 = b, \qquad \mathrm{S}^b_{a+1} = \mathrm{langLift}(\mathrm{S}^b_a), \qquad \mathrm{S}^b_{\lim g}(t) = \lim_i \mathrm{S}^b_{g\,i}(t).$$

With $b = \mathrm{jump}_n$, the first level already contains every System T iteration of the jump along trees, such as transfinite iterates and diagonalizations. The later levels stack the whole language. At input n the tree is $\lim(\mathrm{bootTree}(n),\ \mathrm{S}^{\mathrm{jump}_n}_{\mathrm{bootTree}(n)}(\mathrm{bootTree}(n)))$, and the ladder runs over its addresses. Proved: `contender_36_lt_contender_37`, `family36_lt_F37` and `jtvalue_lt_F37`. The definitions check in 1.9 s and the proofs in 2.5 s.

38 A correction: what a ladder sees

From Contender 15 on, the ladders run over larger and larger well-founded atom orders: ordinal notations, then addresses in collapsed trees. The informal sizes given for these contenders assumed that the ladder's strength grows with the order type of the atoms. That assumption is wrong.

A program of enumeration depth n can mention only the atoms in its enumeration list: notations of size $\le n$, or addresses of depth $\le n$. Every typing rule that involves atoms (`Fits`, `apack`, `aunpack`, kind instantiation with syntactic kinds) refers only to atoms named in the derivation, and erasure forgets all types. So the set of typable erased programs, and with it every ladder value, depends only on the finite order on the named atoms. For address atoms its chains have length at most $n+1$, whatever the tree. The universes over unnamed atoms exist in the semantics, but no program can use them.

The formal results are unaffected. Each comparison theorem is proved, and every contender does exceed the previous one. But from Contender 15 to Contender 37 the growth came from the oracle seeding and the W-ranked amplifiers, which nest each contender's family inside the next. It did not come from the size of the atom order. The trees themselves are genuinely large: they are computed by structural recursion, System T over trees, and System F and ladder programs. They were just used in a place that cannot exploit them. The informal sizes in the status table for Contenders 15–37 describe the atom orders, not the numbers.

39 Contender 38: a fast-growing hierarchy along the trees

Contender 38 uses the tree as the index of a fast-growing hierarchy instead of as ladder atoms. For a base g and a tree t:

$$h^g_0(m) = g(m), \qquad h^g_{t+1}(m) = \big(h^g_t\big)^{m+1}(m), \qquad h^g_{\lim f}(m) = \max_{i \le m} h^g_{f\,i}(m).$$

At limits it takes the maximum over the first $m{+}1$ branches. The usual diagonal $h_{f\,m}(m)$ would be ruined by the zero padding of the program-list limits (`ftTree`, `ladderTree`, `langLift`), because whether a given branch is large depends on where its program falls in the enumeration. With the maximum, every branch $i \le m$ counts. The definition is structural recursion on `CTree`.

The contender is

$$F_{38}(n) = h^{\mathrm{family}_{37}}_{\mathrm{jumpTree}(n)}(\mathrm{family}_{37}(n)), \qquad C_{38} = F_{38}(F_{38}(42)).$$

The base already contains all of Contender 37's amplifiers. The index contains every tree built so far: the collapsing constructions, the stacked tree languages, the System F and ladder program trees, and the jump stacking above them. `le_hier` shows that the hierarchy dominates its base and its argument whenever the base is inflationary. This gives `family37_lt_F38` and `contender_37_lt_contender_38`. The definitions check in 1.6 s and the proofs in 3.1 s.

40 Contender 39: the jump mixed with collapse

In Contender 37 the stacked languages had only the impredicative jump as a primitive, so no level could call Contender 31's predicative operator `boost31`. That operator handles collapsing with tree-indexed levels, universes of codes, and closure functionals of transfinite type. System T over countable trees cannot express it, because collapsing needs higher tree classes. Contender 39's primitive applies both:

$$\mathrm{mixed}_N(t) = \lim\big(\mathrm{jump}_N(t),\ \mathrm{boost}_{31}(\mathrm{jump}_N(t))\big), \qquad \mathrm{mixTree}(N) = \lim\big(J,\ \mathrm{S}^{\mathrm{mixed}_N}_J(J)\big),\ J = \mathrm{jumpTree}(N).$$

Every language level now interleaves the impredicative jump with predicative collapse. The depth parameter of all enumerations inside the tree is $N = \mathrm{family}_{38}(n)$, so the trees contain programs of that size. The number is read off with the hierarchy of Contender 38: $F_{39}(n) = h^{\mathrm{family}_{38}}_{\mathrm{mixTree}(N)}(N)$. Proved: `family38_lt_F39` and `contender_38_lt_contender_39`. The definitions check in 1.1 s and the proofs in 2.6 s.

41 System FT2: an impredicative jump that reads its input

The truncation argument applies to the jump of Contenders 36 and 37 as well. A ladder program over $\mathrm{Addr}(t)$ never inspects t, so the trees it builds are the same for every t. Hence $\mathrm{ladderTree}(t)$ is about $\max(t, C) + 1$ for a constant C, and bootT along an index adds the index's length. The impredicative strength sat in the constant, not in a function of the input.

`SystemFT2` fixes this. It is System F with the tree constructors, a *tree recursor*, and an *oracle tree* T:

$$\mathsf{trec} : \forall X.\ X \to (X \to X) \to ((\mathsf{nat} \to X) \to X) \to \mathsf{tree} \to X, \qquad \mathsf{tin} : \mathsf{tree}.$$

- The recursor reduces on constructor forms: $\mathsf{trec}\,z\,s\,l\,\mathsf{con}_0 \to z$, $\mathsf{trec}\,z\,s\,l\,(\mathsf{con}_1 b) \to s\,(\mathsf{trec}\,z\,s\,l\,b)$, and $\mathsf{trec}\,z\,s\,l\,(\mathsf{con}_2 g) \to l\,(\lambda k.\,\mathsf{trec}\,z\,s\,l\,(g\,k))$.

- The input tree is read lazily by address. $\mathsf{tsub}\,p$ unfolds the node of T at path p into constructor form, and $\mathsf{tsubl}\,p\,\bar{k} \to \mathsf{tsub}(k{::}p)$ gives the branches of a limit node.

Two changes make the candidate proof go through.

- The tree candidate requires the children $g\,a$ of $\mathsf{con}_2\,g$ for every a in the *numeral candidate*, not only for numerals. The limit case of the recursor maps the whole candidate.
- Partial applications of trec (fewer than four arguments) and tsubl are not neutral.

`trec_mem` is proved by induction on the tree candidate nested with SN inductions on the three step functions, and `wt_tsub` by induction on T. Both use only `propext` and `Quot.sound`.

So $\mathrm{jumpF}(t)$, the trees of all programs of depth $\le N$ that read t, contains every tree System F builds from t. It is a genuine operator, and its iterates grow.

42 Contender 40: the jump under first-class language closure

Proposal P27 contributed a *metalanguage*: System T over trees with a closure operator $\mathsf{lift} : (\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree}$ as a primitive (`TreeLanguage4.lean`, adopted). Contender 34 iterated `langLift` externally. Here programs build transformers, close them under the whole lower language, and iterate the closure themselves. `metaBoostT` stacks these metalanguages along a tree, each calling the previous closure as a first-class value.

Contender 40's base transformer joins the FT2 jump, Contender 31's collapse applied to it, and Contender 39's mixed primitive:

$$b_N(t) = \lim\big(\mathrm{jumpF}_N(t),\ \mathrm{boost}_{31}(\mathrm{jumpF}_N(t)),\ \mathrm{mixed}_N(t)\big).$$

The tree is $\lim\big(M,\ \mathrm{metaBoostT}_M(b_N)(M)\big)$ with $M = \mathrm{mixTree}(N)$ and $N = \mathrm{family}_{39}(n)$. The number is $F_{40}(n) = h^{\mathrm{family}_{39}}_{\mathrm{metaTree}(N)}(N)$. Proved: `family39_lt_F40` and `contender_39_lt_contender_40`. The definitions check in 6.5 s, which now includes System FT2, and the proofs in 2.1 s.

43 Contender 41: the ladder jump that reads its input

System FT2 gave System F a recursor over an input tree. `SystemFLT2` gives the same to the ladder language, which has dynamic kinds and a universe for every named atom. The kinds and types are reused from `SystemFL`. Typing, candidates, Church terms and the transfinite universes are copied onto the FT2 terms. As for Contender 36, the new constants need no typing rules. A program of the *reader type*

$$\forall X.\, X \to (X \to X) \to ((\mathsf{nat} \to X) \to X) \to \big(\forall Y.\, Y \to (Y \to Y) \to ((\mathsf{nat} \to Y) \to Y) \to X \to Y\big) \to X \to X$$

is applied, after erasure, to the constructors, the FT2 recursor and the oracle tree. Adequacy with X set to the tree candidate gives `FLT2.wt_of_readerTy`. Two lemmas connect the systems: FL's interpretation of nat has the same members as FT2's numeral candidate (`natP_iff`), and the recursor belongs to the recursor type for every candidate Y (`trec_mem_recTy`).

The ladder jump $\mathrm{jumpL}_N(T)$ joins T with the trees that all reader programs of depth $\le N$ over $\mathrm{Addr}(T)$ build from T. Contender 41 adds it to Contender 40's base, $b'_N(t) = \lim(\mathrm{jumpL}_N(t),\ b_N(t))$, and stacks P27's metalanguages over it to the height of Contender 40's tree:

$$F_{41}(n) = h^{\mathrm{family}_{40}}_{\mathrm{readerTree}(N)}(N), \qquad N = \mathrm{family}_{40}(n).$$

Proved: `family40_lt_F41` and `contender_40_lt_contender_41`. The definitions check in 7.2 s and the proofs in 2.0 s.

44 Contender 42: collapsing relative to the meta-jump

Contenders 40 and 41 iterate the impredicative jumps with languages over *countable* trees. Predicative collapse entered only from outside, as boost_{31} applied to a result, so it never saw how the jump is iterated. `RelCollapse.lean` relativizes Contender 25's Buchholz-style collapse D_v to an operator J: the level-0 successor step becomes

$$D_0^J(t+1) = \lim\big(\varepsilon^+(x),\ J(\varepsilon^+(x))\big), \qquad x = D_0^J(t),$$

with higher levels unchanged. At a supremum, Buchholz's diagonal $x_0 = \Omega_j$, $x_{k+1} = D_j(g\,x_k)$ re-enters level 0, so J is applied at every stage of the collapse, not only to its output. Collapsing an argument above Ω_ω relative to J reaches closure points of J under collapsing, which iteration by a language over countable trees does not.

Contender 42 collapses relative to the *meta-jump* $\mathrm{metaJump}_N(t) = \lim\big(b'_N(t),\ \mathrm{metaBoostT}_t(b'_N)(t)\big)$, the ladder and System F jumps under metalanguages stacked to the height of t. The argument is the ε-iterates above $\Omega_\omega + T$, with T Contender 41's tree:

$$F_{42}(n) = h^{\mathrm{family}_{41}}_{\mathrm{collapseTree}(N)}(N), \qquad N = \mathrm{family}_{41}(n).$$

Proved: `family41_lt_F42` and `contender_41_lt_contender_42`. The definitions check in 1.9 s and the proofs in 1.9 s.

45 Contender 43: collapse as a first-class closure

Contender 42 collapsed relative to one fixed operator. P27's metalanguage takes its closure operator $\mathsf{lift} : (\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree}$ as a parameter, and so far it was language closure (`langLift`). Contender 43's closure also collapses:

$$\mathrm{closeC}_N(J)(t) = \lim\big(\mathrm{langLift}(J)(t),\ \mathrm{relCollapse}(J)(t)\big).$$

`metaBoostC` stacks metalanguages along a tree with this closure at level 0. A level-1 program builds operators from the meta-jump, collapses relative to them, and iterates the results by tree recursion. A level-$(a{+}1)$ program calls the whole closure of level a as a value, so it can collapse relative to collapses. The stack runs to the height of Contender 42's tree, over the meta-jump, applied to that tree:

$$F_{43}(n) = h^{\mathrm{family}_{42}}_{\mathrm{collapseTree43}(N)}(N), \qquad N = \mathrm{family}_{42}(n).$$

Proved: `family42_lt_F43` and `contender_42_lt_contender_43`. The definitions check in 1.4 s and the proofs in 2.2 s.

46 Contender 44: collapsing closures with tree-indexed levels

Contender 43's closure collapses relative to an operator with finitely many levels Ω_k. Contender 26's collapse D^L has levels indexed by trees. `RelLevels.lean` relativizes it to an operator J in the same way: the level-0 successor step applies J, and Buchholz's diagonal at every supremum returns to level 0. One round collapses $\varepsilon_{\Omega_\Lambda+1}$ with the previous tree Λ as the level index:

$$\Lambda_0 = t, \qquad \Lambda_{k+1} = \lim\big(\Lambda_k,\ \psi_0^J(\varepsilon_{\Omega_{\Lambda_k}+1})\big),$$

which climbs toward the fixed points of $\alpha \mapsto \psi_0^J(\Omega_\alpha)$ relative to J. The closure of Contender 44 joins Contender 43's closure with N such rounds started above the finite-level collapse:

$$\mathrm{closeL}_N(J)(t) = \lim\big(\mathrm{closeC}_N(J)(t),\ \Lambda_N^J(\mathrm{relCollapse}(J)(t))\big).$$

Metalanguages with this closure are stacked to the height of Contender 43's tree over the meta-jump, and $F_{44}(n) = h^{\mathrm{family}_{43}}_{\mathrm{levelTree44}(N)}(N)$ with $N = \mathrm{family}_{43}(n)$. Proved: `family43_lt_F44` and `contender_43_lt_contender_44`. The definitions check in 2.3 s and the proofs in 3.0 s.

47 Contender 45: the construction as its own jump

Contenders 40–44 build trees with operators over a *fixed* base, the meta-jump. But each construction is itself an operator on trees, so it can serve as the base of the next one:

$$\Theta_0 = \mathrm{metaJump}_N, \qquad \Theta_{a+1}(t) = \mathrm{metaBoostC}(\mathrm{closeL}_N)\,(t)\,(\Theta_a)\,(t), \qquad \Theta_{\lim f}(t) = \lim_i \Theta_{f\,i}(t).$$

At each successor stage, collapsing metalanguages are stacked to the height of the argument over the whole previous stage, and then applied to the argument. Θ is iterated along Contender 44's tree and applied to it: $F_{45}(n) = h^{\mathrm{family}_{44}}_{\mathrm{thetaTree}(N)}(N)$ with $N = \mathrm{family}_{44}(n)$. Proved: `family44_lt_F45` and `contender_44_lt_contender_45`. The definitions check in 1.3 s and the proofs in 1.9 s.

48 Contender 46: universes collapsed relative to the closure's operator

Contenders 27–31 build universes of level codes, universe operators and closure functionals of transfinite type. Until now they entered only through boost_{31}, which finishes by collapsing $\varepsilon_{\Omega_{\mathrm{top}}+1}$ of the next universe with fixed ε-steps. The countable level of that collapse is exactly the countable trees, so the collapse relativizes in the same way as D and D^L (`RelUniverses.lean`): at level 0 the successor step is $\lim(\varepsilon^+x,\ J(\varepsilon^+x))$. $\mathrm{boost}_{31}^J(\Lambda)$ is Contender 31's construction over the index Λ, collapsed relative to J. The first-class closure becomes

$$\mathrm{closeU}_N(J)(t) = \lim\big(\mathrm{closeL}_N(J)(t),\ \mathrm{boost}_{31}^J(t),\ \mathrm{boost}_{31}^J(\Lambda_N^J(t))\big),$$

so programs can collapse whole universes relative to operators they build. Contender 45's self-based iteration Θ is rerun with this closure along Contender 45's tree: $F_{46}(n) = h^{\mathrm{family}_{45}}_{\mathrm{univTree46}(N)}(N)$ with $N = \mathrm{family}_{45}(n)$. Proved: `family45_lt_F46` and `contender_45_lt_contender_46`. The definitions check in 2.0 s and the proofs in 2.4 s.

49 Contender 47: iterating the self-based iteration

Contender 45's Θ starts from the meta-jump and makes each stage the base of the next. Started from an arbitrary base B (with Contender 46's closure), the iteration is again an operator on trees, written Θ^B. It can itself be iterated:

$$\Psi_0 = \mathrm{metaJump}_N, \qquad \Psi_{a+1}(t) = \Theta_t^{\Psi_a}(t), \qquad \Psi_{\lim f}(t) = \lim_i \Psi_{f\,i}(t).$$

Ψ_{a+1} runs the whole self-based iteration along its argument, starting from all of Ψ_a. Ψ is iterated along Contender 46's tree: $F_{47}(n) = h^{\mathrm{family}_{46}}_{\mathrm{psiTree}(N)}(N)$ with $N = \mathrm{family}_{46}(n)$. Proved: `family46_lt_F47` and `contender_46_lt_contender_47`. The definitions check in 1.5 s and the proofs in 2.1 s.

50 Contender 48: a tree-indexed hierarchy of iteration levels

Contender 45's Θ (a base turned into an iteration along a tree) and Contender 47's Ψ are the first two levels of a pattern. For any Φ that turns a base into an iteration, iterOf(Φ) iterates "run Φ from the previous stage":

$$\mathrm{iterOf}(\Phi)(B)_0 = B, \qquad \mathrm{iterOf}(\Phi)(B)_{a+1}(t) = \Phi\big(\mathrm{iterOf}(\Phi)(B)_a\big)_t(t),$$

with pointwise limits, so $\Psi = \mathrm{iterOf}(\Theta)(\mathrm{metaJump})$. `iterLevel` indexes these levels by a tree: level 0 is Θ, level $k{+}1$ is iterOf of level k, and limit levels are pointwise limits. This is a Veblen-style hierarchy of iteration operators. The level index, the iteration height and the argument are all Contender 47's tree: $F_{48}(n) = h^{\mathrm{family}_{47}}_{\mathrm{levelIterTree}(N)}(N)$ with $N = \mathrm{family}_{47}(n)$. Proved: `family47_lt_F48` and `contender_47_lt_contender_48`. The definitions check in 1.5 s and the proofs in 4.4 s.

51 Contender 49: impredicative reflection over the tree

Since Contender 38 the trees have been turned into numbers by h, one fixed fast-growing hierarchy. System FT2 reads a tree far more strongly. Its programs of type nat see the tree as the oracle tree and recurse over it into every type, impredicatively. They can implement h itself, and also iterate along the tree at higher types, build impredicative encodings over it, and diagonalize. The reflection

$$R^{g,T}(m) = m + 1 + \max\big(g(m),\ \max\{\,\mathrm{val}_{g,T}(p) : p \text{ an FT2 program of type nat and depth} \le m\,\}\big)$$

collects them, and Contender 49 runs the hierarchy along Contender 48's tree with this reflection as its base:

$$F_{49}(n) = h^{R^{\mathrm{family}_{48},T}}_T(N), \qquad N = \mathrm{family}_{48}(n),\ T = \mathrm{levelIterTree}(N).$$

Proved: `natValue_lt_treeReflect` (every such program is below the reflection), `family48_lt_F49` and `contender_48_lt_contender_49`. The definitions check in 1.0 s and the proofs in 3.6 s.

52 Contender 50: ladder reflection over the tree

Contender 49 read its tree with System FT2 programs. The ladder language is stronger: it has dynamic kinds and a universe for every named atom. A ladder program of the *numeric reader type*

$$\forall X.\ \big(\forall Y.\ Y \to (Y \to Y) \to ((\mathsf{nat} \to Y) \to Y) \to X \to Y\big) \to X \to \mathsf{nat}$$

is applied, after erasure, to the FT2 recursor and the oracle tree. Adequacy with X set to the tree candidate puts the result in the interpretation of nat, so it is strongly normalizing (`FLT2.sn_of_natReaderTy`), and its normal form is read off as a numeral (`FLT2.readNat`). The reflection $L^{g,T}(m)$ exceeds Contender 49's reflection and every such program of depth $\le m$ over Addr(T), and

$$F_{50}(n) = h^{L^{\mathrm{family}_{49},T}}_T(N), \qquad N = \mathrm{family}_{49}(n),\ T = \mathrm{levelIterTree}(N).$$

Proved: `readNat_lt_ladderReflect`, `family49_lt_F50` and `contender_49_lt_contender_50`. The definitions check in 3.2 s and the proofs in 2.3 s.

53 Contender 51: programs over the iteration operators

Contenders 45–48 apply the iteration operators along trees chosen from outside. `PrimLanguage.lean` generalizes System T over trees to an arbitrary typed environment of primitives: terms refer to primitives by index, and the interpreter is still structural recursion on typed terms. Contender 51's primitives are the seed tree (Contender 48's tree), the meta-jump, the universe-collapsing closure, and the iteration operators

$$\Theta : \Phi, \qquad \mathrm{iterOf} : \Phi \to \Phi, \qquad \Phi = (\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree} \to \mathsf{tree}.$$

The level hierarchy of Contender 48 is then a program, iterLevel = $\mathsf{treeRec}\ \Theta$ iterOf (pointwise limit), which programs can apply at levels, heights and arguments they compute themselves, nested and diagonalized at higher types. The tree joins all closed tree programs of every size, and the number is read off by Contender 50's ladder reflection along it. Proved: `family50_lt_F51` and `contender_50_lt_contender_51`. The definitions check in 3.9 s and the proofs in 3.3 s.

54 Contender 52: languages over the iteration operators, stacked

Contender 51's language is itself a closure operator: $\mathrm{ptlLift}_{P,\sigma}(b)(t)$ applies every closed $\mathsf{tree} \to \mathsf{tree}$ program over the primitives P plus b to t. `ptlStack` stacks these languages along a tree, as P27 did for System T over a single primitive. Each successor level adds the previous level's closure as a first-class primitive $\mathsf{lift} : (\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree}$, and limits are pointwise. With Contender 51's primitives (the seed, the meta-jump, the universe-collapsing closure and the iteration operators), the stack runs to the height of Contender 51's tree over the meta-jump: $F_{52}(n) = h_T^{L^{\mathrm{family}_{51},T}}(N)$ with $N = \mathrm{family}_{51}(n)$ and $T = \mathrm{stackTree}(N)$. Proved: `family51_lt_F52` and `contender_51_lt_contender_52`. The definitions check in 1.7 s and the proofs in 2.7 s.

55 Contender 53: reflection iterated along the tree

Contenders 49–52 read the final tree with one reflection, whose programs recurse over the tree with a fixed number oracle. Contender 53 iterates the reflection along the tree itself:

$$\rho_0 = g, \qquad \rho_{a+1} = L^{\rho_a,T}, \qquad \rho_{\lim f}(m) = \max_{i \le m} \rho_{f\,i}(m).$$

At a successor level, every FT2 and ladder program reading T may call the previous level as its number oracle. So each level closes the previous one under all programs over T. This is hyperreflection (Contenders 17 and 23) along the giant tree. `le_reflAlong` shows that every level dominates the base and the argument, and $F_{53}(n) = h_T^{\rho_T}(N)$ with $g = \mathrm{family}_{52}$, $N = \mathrm{family}_{52}(n)$ and $T = \mathrm{stackTree}(N)$. Proved: `family52_lt_F53` and `contender_52_lt_contender_53`. The definitions check in 1.6 s and the proofs in 4.7 s.

56 Contender 54: reflection as a primitive

Contender 53 iterates ladder reflection along one fixed tree, which is one particular recursion. Contender 54 makes reflection a primitive of Contender 51's language over the iteration operators:

$$\mathsf{reflT} : \mathsf{tree} \to (\mathsf{nat} \to \mathsf{nat}) \to \mathsf{nat} \to \mathsf{nat}, \qquad \mathsf{reflT}\ t\ g = L^{g,t},$$

next to the number oracle g and all of Contender 51's tree primitives. Closed programs of type nat can build trees with the jumps, collapses and iteration operators, reflect over *any* tree they build, and iterate this at higher types. Contender 53's ρ is one such program. The reflection $\mathrm{numReflect}_N(g)(m)$ exceeds $g(m)$ and every closed nat program of size $\le m$ (`program_lt_numReflect`), and $F_{54}(n) = h_T^{\mathrm{numReflect}_N(\mathrm{family}_{53})}(N)$ with $N = \mathrm{family}_{53}(n)$ and $T = \mathrm{stackTree}(N)$. Proved: `family53_lt_F54` and `contender_53_lt_contender_54`. The definitions check in 1.9 s and the proofs in 2.5 s.

57 Contender 55: number languages stacked along the tree

Contender 54 closes a number oracle once under every program over its primitives, which include reflection over program-built trees. Contender 55 stacks such closures along a tree, as `ptlStack` stacks the tree languages:

$$\mathrm{numLift}_P(g)(m) = m + 1 + \max\big(g(m),\ \max\{\, v : v \text{ a closed nat program over } P + g \text{ of size} \le m \,\}\big),$$

$$S_0 = \mathrm{numLift}_P, \qquad S_{a+1} = \mathrm{numLift}_{P+[\mathsf{lift} := S_a]}, \qquad S_{\lim f}(g)(m) = \max_{i \le m} S_{f\,i}(g)(m).$$

Each successor level can call the previous level's whole closure as a first-class $\mathsf{lift} : (\mathsf{nat} \to \mathsf{nat}) \to \mathsf{nat} \to \mathsf{nat}$, and iterate and compose it at higher types. `lt_numStack` shows that every level strictly dominates its oracle and argument, with no assumption on the oracle. The stack runs along Contender 52's stacked tree T with Contender 54's primitives, and $F_{55}(n) = h_T^{S_T(\mathrm{family}_{54})}(N)$ with $N = \mathrm{family}_{54}(n)$. Proved: `family54_lt_F55` and `contender_54_lt_contender_55`. The definitions check in 1.8 s and the proofs in 2.3 s.

58 Contender 56: tree and number languages stacked jointly

Contenders 52 and 55 stack tree languages and number languages separately. The number programs of Contender 55 build trees only with Contender 51's fixed operators, never with the stacked tree languages. `jointStack` stacks the two kinds of closure together. At each successor level, the language has as primitives *both* closures of the previous level:

$$\mathsf{liftT} : (\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree}, \qquad \mathsf{liftN} : (\mathsf{nat} \to \mathsf{nat}) \to \mathsf{nat} \to \mathsf{nat}.$$

So a number program can build trees with the whole previous tree language, reflect over them with reflT, and close number functions under the previous number language. Limits are pointwise for trees and maxima for numbers. The joint stack first builds a tree T' along Contender 52's tree over the meta-jump, then runs along T'. Its number closure is the base of the hierarchy: $F_{56}(n) = h_{T'}^{S'_{T'}(\mathrm{family}_{55})}(N)$ with $N = \mathrm{family}_{55}(n)$. `lt_jointStack` shows that every level strictly dominates its oracle and argument. Proved: `family55_lt_F56` and `contender_55_lt_contender_56`. The definitions check in 1.6 s and the proofs in 2.8 s.

59 Contender 57: stacking heights chosen by programs

Contender 56's joint stack runs to heights fixed from outside. Contender 57 makes it a primitive at any height a program computes:

$$\mathsf{stackT} : \mathsf{tree} \to (\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree}, \qquad \mathsf{stackN} : \mathsf{tree} \to (\mathsf{nat} \to \mathsf{nat}) \to \mathsf{nat} \to \mathsf{nat},$$

the tree and number closures of the joint stack at height t. These sit next to Contender 54's primitives (tree operators, iteration operators, reflection, the number oracle). Programs build a height, for instance from another stack's output, stack languages to it, and repeat this at higher types. The tree joins Contender 56's tree with the closure of the meta-jump under this language, and the language's number closure is the hierarchy's base: $F_{57}(n) = h_T^{\mathrm{numLift}(\mathrm{family}_{56})}(N)$ with $N = \mathrm{family}_{56}(n)$ and $T = \mathrm{heightTree}(N)$. Proved: `family56_lt_F57` and `contender_56_lt_contender_57`. The definitions check in 1.5 s and the proofs in 2.6 s.

60 Contender 58: a tower of stack-reflecting languages

Contender 57 turns the joint stack over a language into two primitives of a larger language. The step can be repeated. Level 0 has Contender 54's primitives, and level $k{+}1$ adds stackT and stackN for the joint stack *over the language of level* k, at program-chosen heights:

$$P_0 = P_{54}, \qquad P_{k+1} = \mathsf{stackT}_k :: \mathsf{stackN}_k :: P_k.$$

A program at level $k{+}1$ stacks, to heights it chooses, languages that themselves stack languages, k levels deep. The tower height is $N = \mathrm{family}_{57}(n)$. The tree joins Contender 57's tree with the closure of the meta-jump under the top language, and the top language's number closure is the hierarchy's base: $F_{58}(n) = h_T^{\mathrm{numLift}_{P_N}(\mathrm{family}_{57})}(N)$. Proved: `family57_lt_F58` and `contender_57_lt_contender_58`. The definitions check in 1.2 s and the proofs in 2.0 s.

61 Contender 59: the tower of stack-reflecting languages along a tree

Contender 58's tower has a natural-number height, because primitive lists are finite. Working with closure pairs instead of primitive lists, the tower can be indexed by a tree. `jointFrom` is Contender 56's joint stack started from a given closure pair c. `towerStep` closes the language whose primitives are Contender 54's plus the height-indexed joint stacks started from c, and `towerT` iterates it:

$$C_0 = \text{closure of } P_{54}, \qquad C_{a+1} = \mathrm{towerStep}(C_a), \qquad C_{\lim f} = \text{pointwise limit and maximum.}$$

The tower runs along Contender 58's tree to build the tree T, then along T for the number closure: $F_{59}(n) = h_T^{C_T.2(\mathrm{family}_{58})}(N)$ with $N = \mathrm{family}_{58}(n)$. `lt_towerT` shows that every level strictly dominates its oracle and argument. Proved: `family58_lt_F59` and `contender_58_lt_contender_59`. The definitions check in 1.1 s and the proofs in 2.0 s.

62 Contender 60: a hierarchy of tower operators

Contender 59's tower is a height-indexed family of closure pairs. A tower can carry such a family H as extra primitives: `towerH` is Contender 59's tower whose languages also contain $H(t)$ for every program-chosen height t. Iterating $H \mapsto \mathrm{towerH}(H)$ along a tree gives a hierarchy of tower operators:

$$U_0 = \mathrm{towerT}, \qquad U_{k+1} = \mathrm{towerH}(U_k), \qquad U_{\lim f} = \text{pointwise.}$$

At level $k{+}1$ the towers' languages contain the level-k towers at every height. The level, the height and the argument are all Contender 59's tree. The result builds a tree T, and the number closure

at level and height T is the hierarchy's base: $F_{60}(n) = h_T^{U_T(T).2(\mathrm{family}_{59})}(N)$ with $N = \mathrm{family}_{59}(n)$. `lt_upLevels` shows that every level and height strictly dominates its oracle and argument. Proved: `family59_lt_F60` and `contender_59_lt_contender_60`. The definitions check in 1.0 s and the proofs in 1.9 s.

63 Contender 61: programs over the tower operators

Contenders 57–60 each turn one construction into a primitive, or iterate it along a tree chosen from outside. Closure operators on tree operators and on number functions come in pairs, so a language that handles them as values needs products. `PrimLanguage2.lean` adds them: PTL2 is System T over trees with products, pairing and projections, and an environment of primitives. Contender 61's primitives are the seed tree, the meta-jump, the number oracle, reflection reflT, and the tower operators

$$\mathsf{towerT} : \mathsf{tree} \to C, \qquad \mathsf{towerH} : (\mathsf{tree} \to C) \to \mathsf{tree} \to C, \qquad C = ((\mathsf{tree} \to \mathsf{tree}) \to \mathsf{tree} \to \mathsf{tree}) \times ((\mathsf{nat} \to \mathsf{nat}) \to \mathsf{n}$$

So Contender 60's $U_k = \mathsf{treeRec}\ \mathsf{towerT}\ \mathsf{towerH}$ (limit) k is a program, and programs can apply towerH to families they build themselves, at levels and heights they compute, at higher types. The tree joins Contender 60's tree with the meta-jump's closure under this language, and the language's number closure is the hierarchy's base: $F_{61}(n) = h_T^{\mathrm{num2Lift}(\mathrm{family}_{60})}(N)$ with $N = \mathrm{family}_{60}(n)$. Proved: `program_lt_num2Lift`, `family60_lt_F61` and `contender_60_lt_contender_61`. The definitions check in 3.0 s and the proofs in 2.7 s.

64 Contender 62: the tower language stacked along a tree

Contender 61's language has products, so a whole closure pair C is a single value. `stack2` stacks the language along a tree. Level 0 is the language's closure pair. Level $a{+}1$'s language has level a's closure pair as one extra primitive of type C, so programs can apply both closures, compose them and iterate them at higher types. Limits are pointwise for trees and maxima for numbers. The stack runs along Contender 61's tree to build a tree T, then along T for the number closure: $F_{62}(n) = h_T^{S_T^2.2(\mathrm{family}_{61})}(N)$ with $N = \mathrm{family}_{61}(n)$. `lt_stack2` shows that every level strictly dominates its oracle and argument. Proved: `family61_lt_F62` and `contender_61_lt_contender_62`. The definitions check in 1.2 s and the proofs in 2.0 s.

65 Contender 63: programs over the stacking operator of the tower language

Contender 62 stacks Contender 61's language along a tree. With products, the steps of Contenders 59–61 (stack to any height, add a height-indexed family of closures as primitives, make the operator a value) fit in one language. Its two new primitives are

$$\mathsf{stack2} : \mathsf{tree} \to C, \qquad \mathsf{up2} : (\mathsf{tree} \to C) \to \mathsf{tree} \to C,$$

where $\mathsf{up2}(H)$ is the stack whose languages also contain the family H. Hierarchies such as $\mathsf{up2}(\mathsf{up2}(\cdots\mathsf{stack2}))$, iterated along trees that programs build, are therefore programs. The tree joins Contender 62's tree with the meta-jump's closure under this language, and the language's number closure is the hierarchy's base: $F_{63}(n) = h_T^{\mathrm{num2Lift}(\mathrm{family}_{62})}(N)$ with $N = \mathrm{family}_{62}(n)$. Proved:

`family62_lt_F63` and `contender_62_lt_contender_63`. The definitions check in 2.7 s and the proofs in 2.7 s.

66 Contender 64: the stacking-operator step, towered along a tree

Contender 63 turns Contender 62's stack and its family-raising operator into values of a larger language. That step can be repeated, and with closure pairs the repetitions can be indexed by a tree. `stack2From` is Contender 62's stack started from a given closure pair c. One step,

$$\text{step2}(c) = \text{closures of } P_{61} + \big[\mathsf{stack2}_c : \mathsf{tree} \to C,\ \ \mathsf{up2}_c : (\mathsf{tree} \to C) \to \mathsf{tree} \to C\big],$$

closes Contender 61's language plus the stack started from c and the operator raising families to stacks started from c. `tower2` iterates it along a tree, pointwise at limits. The tower runs along Contender 63's tree to build a tree T, then along T for the number closure: $F_{64}(n) = h_T^{\text{tower2}_T.2(\text{family}_{63})}(N)$ with $N = \text{family}_{63}(n)$. `lt_tower2` shows that every level strictly dominates its oracle and argument. Proved: `family63_lt_F64` and `contender_63_lt_contender_64`. The definitions check in 1.3 s and the proofs in 2.1 s.

67 Contender 65: a hierarchy of step operators

Contenders 57–64 follow one pattern. A *step* S maps a closure pair to the next one. towerFrom$(S)(c)$ iterates it along a tree starting from c. The tower then becomes a value of a new language, which gives a new step. Contender 65 turns the pattern itself into a hierarchy:

$$\text{metaStep}(S)(c) = \text{closures of } P_{61} + \big[\text{towerFrom}(S)(c) : \mathsf{tree} \to C\big],$$

$$M_0 = \text{step2}, \qquad M_{a+1} = \text{metaStep}(M_a), \qquad M_{\lim f} = \text{pointwise}.$$

M_a is the pattern applied a times, for a transfinite tree a. The proofs use one invariant. A closure pair is *good* when its number closure strictly dominates its oracle and argument. Every step in the hierarchy produces good pairs (`good_metaS`), and towers of good steps from good pairs stay good (`good_towerFrom`). The meta-level, the tower height and the argument are Contender 64's tree, then the tree this builds: $F_{65}(n) = h_T^{\text{towerFrom}(M_T)(\cdot)(T).2(\text{family}_{64})}(N)$ with $N = \text{family}_{64}(n)$. Proved: `family64_lt_F65` and `contender_64_lt_contender_65`. The definitions check in 1.4 s and the proofs in 2.5 s.

68 Contender 66: programs over closure and towering

Contender 65's hierarchy of step operators is built from two operations: closing a language with an extra family of closures, and towering a step along a tree. Contender 66 makes both values of the language with products:

$$\mathsf{closeWith} : (\mathsf{tree} \to C) \to C, \qquad \mathsf{towerFrom} : (C \to C) \to C \to \mathsf{tree} \to C.$$

$\mathsf{closeWith}(H)$ is the closure pair of Contender 61's language plus the family H. Then metaStep$(S)(c) = \mathsf{closeWith}(\mathsf{towerFrom}\ S\ c)$ is a program, Contender 65's hierarchy M is a tree recursion at type $C \to C$, and programs can diagonalize across meta-levels, feed towers of one hierarchy into another, and iterate at higher types. These primitives join Contender 61's. The tree joins Contender 65's tree with the meta-jump's closure under this language, and the language's number closure is the hierarchy's base: $F_{66}(n) = h_T^{\text{num2Lift}(\text{family}_{65})}(N)$ with $N = \text{family}_{65}(n)$. Proved: `family65_lt_F66` and `contender_65_lt_contender_66`. The definitions check in 1.7 s and the proofs in 3.0 s.

69 Contender 67: a reflective tower of closure operators

Contender 66's closeWith closes the *base* language with a family of closures. Contender 67 raises the closing operator itself along a tree:

$$cw_0(H) = \text{closures of } P_{66} + [H], \qquad cw_{a+1}(H) = \text{closures of } P_{66} + [H,\ cw_a : (\mathsf{tree} \to C) \to C],$$

with pointwise limits. The primitive types are the same at every successor level, so one recursion on the operator suffices. A program at level $a{+}1$ can close level a's language with families it builds, including towers of steps built from cw_a itself. Stacking (Contender 62) is the special case of a constant family. Every level is a good closure pair for every family (`good_cwTower`). The tower runs to the height of Contender 66's tree to build a tree T, then to height T for the number closure: $F_{67}(n) = h_T^{cw_T(\cdot).2(\text{family}_{66})}(N)$ with $N = \text{family}_{66}(n)$. Proved: `family66_lt_F67` and `contender_66_lt_contender_67`. The definitions check in 1.8 s and the proofs in 2.5 s.

70 Contender 68: programs over the reflective tower's step

Contender 67's tower of closure operators has a uniform step: $\text{cwStep}(X)(H)$ is the closure of Contender 66's language plus H and the operator X, and $cw_{a+1} = \text{cwStep}(cw_a)$. Contender 68 makes the step and the tower values of the language:

$$\mathsf{cwStep} : cwTy \to cwTy, \qquad \mathsf{cwAt} : \mathsf{tree} \to cwTy, \qquad cwTy = (\mathsf{tree} \to C) \to C.$$

Programs iterate cwStep from any operator they build, not only from cw_0, along trees they build. They also diagonalize, and feed one tower into another. These primitives join Contender 66's. The tree joins Contender 67's tree with the meta-jump's closure under this language, and the language's number closure is the hierarchy's base: $F_{68}(n) = h_T^{\text{num2Lift}(\text{family}_{67})}(N)$ with $N = \text{family}_{67}(n)$. Proved: `family67_lt_F68` and `contender_67_lt_contender_68`. The definitions check in 1.9 s and the proofs in 3.5 s.

71 Contender 69: the reflective tower over the step language

Contender 67 raised the closure operator of Contender 66's language along a tree. Contender 68's language is stronger, because it contains that tower's step and the tower itself as values. `cwTowerP` is Contender 67's construction for any base language (P, σ):

$$cw_0^P(H) = \text{closures of } P + [H], \qquad cw_{a+1}^P(H) = \text{closures of } P + [H,\ cw_a^P],$$

with pointwise limits. Contender 69 runs it over Contender 68's language, along Contender 68's tree, then along the tree this builds: $F_{69}(n) = h_T^{cw_T^{P_{68}}(\cdot).2(\text{family}_{68})}(N)$ with $N = \text{family}_{68}(n)$. `good_cwTowerP` shows that every level is a good closure pair for every family. Proved: `family68_lt_F69` and `contender_68_lt_contender_69`. The definitions check in 2.6 s and the proofs in 4.5 s.

72 Contender 70: the reflection cycle repeated

Contenders 67 and 68 form a cycle. First a language's closure operator is raised along a tree (`cwTowerP`). Then that tower's step and the tower itself become values, which gives a stronger language. Contender 69 started the cycle again. Contender 70 repeats it N times:

$$P_0^R = P_{66}, \qquad P_{k+1}^R = \mathsf{cwStep}_k :: \mathsf{cwAt}_k :: P_k^R,$$

where cwStep_k and cwAt_k are the step and the tower of the reflective tower over the level-k language. The first cycle reproduces Contender 68's language. The reflective tower over the top language runs along Contender 69's tree to build a tree T, then along T for the number closure: $F_{70}(n) = h_T^{cw_T^{P^R_N}(\cdot).2(\mathrm{family}_{69})}(N)$ with $N = \mathrm{family}_{69}(n)$. Proved: `good_cycleTower`, `family69_lt_F70` and `contender_69_lt_contender_70`. The definitions check in 1.6 s and the proofs in 4.9 s.

73 An independent tower

From Contender 7 on, almost every contender takes its predecessor's family as an oracle or seed. This makes each comparison short (the new family dominates its own seed), but it also makes the tower cumulative: the size of Contender 70 is the size of a long composition, and the article cannot say which ideas carry it. The *independent tower* (`BusyLean/Independent`) is built under one extra rule: **no rung uses a smaller rung in its definition**. Each rung is a single idea, and its comparison with the rung below is a simulation: the larger construction runs the smaller one. When a construction would naturally be built on a smaller rung, the smaller rung is dropped. The tower has three rungs.

Rung	idea	definition
1	nested recursion on numbers	`ack 5 42 10002` (Contender 4 of the Coq contest)
2	busy beaver of System T	largest numeral of depth 42 (Contender 5 of the Coq contest)
3	impredicative universes, self-reflected	`F3 (F3 42)` below

Rungs 1 and 2. The two Coq contenders are already independent of each other. The Coq proof of `contender_4_lt_contender_5` writes Ackermann's function as a System T term; it is ported to Lean as `contender_1_lt_contender_2`. The only subtlety is to keep Lean's kernel from evaluating `ack 5 42 10002`: every equation is stated for symbolic arguments and instantiated afterwards.

Rung 3. The language is the ladder of universes of Contender 14, used as a library, with its oracle constant pointed at the construction itself:

```
def ladderMax (g : ℕ → ℕ) (n : ℕ) : ℕ :=
  n + 1 + listMax (fvalue n g n) (ladderUpTo n)
def stage : ℕ → ℕ → ℕ
  | 0 => Nat.succ
  | k + 1 => ladderMax (stage k)
def F3 (n : ℕ) : ℕ := stage (n + 1) n
def contender_3 : ℕ := F3 (F3 42)
```

Stage $k+1$ maximizes level-n programs of depth $\le n$ whose oracle is stage k. `F3` diagonalizes over reflection stages, universe levels and program sizes at once. The enumeration and the value function are redefined in the rung's own module; the audit (`test/IndependentAudit.lean`) lists the modules of the definition's closure (the ladder library and System F's core) and rejects any contender module.

Rung 2 < rung 3: compiling System T. The winner of rung 2 is an arbitrary raw term of depth ≤ 42, possibly ill-typed, and Contender 5 interprets such terms with casts and default values. The proof therefore compiles *every* raw term.

1. **Static types.** The type tag of a term's value depends only on the term and the types of its environment (`interp_fst`). The translation uses it to resolve Contender 5's quirks statically: a cast between different types becomes the default value, an application of a number becomes $\overline{0}$.

2. **Simulation.** Natural recursion becomes iteration over polymorphic pairs. A logical relation (at `nat`: "reduces to the Church numeral") and its fundamental lemma `rel_utr` show that a closed term of type `nat` reduces to the numeral of its value, under any oracle.

3. **Confluence.** The normalizer follows normal order, while the simulation uses arbitrary β-steps. Takahashi's complete development proves Church–Rosser for β plus the oracle rule (`nf_eq_of_steps`). The oracle fires only on literal numerals, which are normal, so it does not interfere.

4. **Typing.** The Church-style translation lies in the System F fragment of the ladder language (no kinds but $\star$, no atoms). The ladder's type checker accepts it at every fuel (`infer_trC`).

5. **Size.** A depth-42 term compiles to depth at most 186 (`cdepth_trC`). The compiled program $\texttt{natRec}\,\texttt{succ}\,(\lambda_\, g\, x.\, g\,(g\, x))\,10\,0 = 2^{10}$ fits in depth 42, so `F3 42` ≥ 1067, and the outer `F3` sees the compiled winner.

The whole check takes about 11 s.

What was left out, and why. System F, F_ω, kind polymorphism and the dynamic-kind universe are fragments of the ladder language, and the W-language and the tree languages extend System T, so each would be incorporated by a larger rung. Reflection along trees, towers, closure operators and reflective towers (Contenders 6, 9–12, 15, 23 and 51–70) are amplifiers of a base language; one of them, self-reflection, appears inside rung 3. Ordinal-notation hierarchies (Contenders 16–31) are excluded for another reason. Placed above System T they would need an upper bound on System T's values, that is, an ordinal analysis. Placed below it they would merely generalize Ackermann.

74 Roadmap

The construction now has three layers. Their weak points mark where further growth must come from.

Languages that read trees. The impredicative languages (System FT2 and the ladder language FLT2) recurse over an input tree at every type. They build trees from it (the jumps of Contenders 40–41) and compute numbers from it (the reflections of Contenders 49–50). Their strength is fixed by the language: System F, or the ladder with a universe for each named atom. The next step up is a stronger reading language. One option is type-level recursion over the input tree, so that universes are indexed by the tree's nodes and not just by named atoms. This is large elimination, the route to a CIC-like system, and it needs a new strong normalization proof.

Using the atom order. Section 38 showed that a ladder sees only the atoms it names. Quantification over atoms, such as a type former $\forall b < a.\, A$ interpreted as an intersection over all atoms below a, would let programs act uniformly across the transfinite order. Whether that adds strength without recursion over atoms is open. Atom recursion would.

Operators on trees. Contenders 42–48 stack operators on trees: collapsing relative to a jump, collapse as a first-class closure, the construction as its own base, and a Veblen-style hierarchy of iteration levels. The natural continuation is collapsing at this level, with iteration levels indexed by higher tree classes and then collapsed, as Buchholz's functions collapse the levels Ω_ν. A complementary step is a metalanguage whose primitives include the iteration operators themselves, so that programs can iterate them at higher types.

Reading the final tree. Contenders 49 and 50 turn the final tree into a number with programs that recurse over it. Any stronger reading language from the first paragraph improves this step directly.

Ladder lengths. The ordinal-notation ladders of Contenders 15–24 remain correct and well-founded, but by Section 38 their length does not reach the numbers. Their well-foundedness proofs are reusable wherever a genuine transfinite recursion is needed.

75 Status

Contender	idea	informal size
5 (Coq)	System T, depth 42	$f_{<\varepsilon_0}(42)$
6 (Lean)	W-language, reflected along its own trees	$f_{\psi(\Omega_\omega)^2}(42)$
7 (Lean)	System F with Contender 6 as oracle	$f_{\lvert \mathsf{Z}_2 \rvert}(>C_6)$
8 (Lean)	System F_ω with Contender 7 as oracle	$f_{\lvert \mathsf{Z}_\omega \rvert}(>C_7)$
9 (Lean)	F_ω reflected along W-trees	$f_{\lvert \mathsf{Z}_\omega \rvert \cdot \psi(\Omega_\omega)^2}(>C_8)$
10 (Lean)	kind-polymorphic F_ω	beyond higher-order arithmeti
11 (Lean)	10, reflected along W-trees	multiplicative gain over 10
12 (Lean)	10, operator-level reflection (P6, P7 ideas)	exponential index over 11
13 (Lean)	dynamic kind: a universe of operators	uniform orders $\omega + n$
14 (Lean)	ladder of universes, level n at input n	orders $\omega \cdot n$, diagonalized
15 (Lean)	14 with P17's rank hierarchy of lifts	$\lvert \mathrm{FL}_n \rvert \cdot \varepsilon(\tau)$
16 (Lean)	15 over a ladder of length ω^ω	levels $< \omega^n$ at input n
17 (Lean)	15 over a ladder of length ε_0	levels $< \omega_n$ (towers) at input
18 (Lean)	15 over a ladder of length Γ_0	φ-towers of height n at input
19 (Lean)	15 over a ladder of large-Veblen length	Klammersymbole of height n
20 (Lean)	15 over a ladder of Bachmann–Howard length	ψ-terms of height n at input n
21 (Lean)	15 over Buchholz ladders, level bound n at input n	lengths up to $\psi_0(\Omega_\omega)$
22 (Lean)	15 over a tower of relativized Buchholz notations	beyond $\psi_0(\Omega_\omega)$
23 (Lean)	22 with P22's W-ranked lift hierarchy	transfinite rank over 22
24 (Lean)	towers along well-founded indices, self-indexed rounds	fixed points of $x \mapsto \psi_0(\Omega_\omega x)$
25 (Lean)	ladders over addresses in collapsed tree ordinals	$\psi_0(\varepsilon_{\Omega_\omega + n})$ at input n
26 (Lean)	collapsing with tree-indexed levels	toward fixed points of $\alpha \mapsto \psi_0$
27 (Lean)	universes of level codes	toward the first fixed point of
28 (Lean)	universes stacked along collapsed trees	iterated universe heights
29 (Lean)	higher-order universe operators along trees	hierarchy of inaccessible-like c
30 (Lean)	closure functionals of all finite types	higher-type closure hierarchies
31 (Lean)	closure functionals of transfinite (tree-indexed) type	type height = collapsed tree
32 (Lean)	System T over trees with Contender 31 as a primitive	all programs of size $\leq n{+}3$
33 (Lean)	the language stacked $n{+}1$ times on itself	languages over languages
34 (Lean)	the language stacked along a collapsed tree	stacking height = tree
35 (Lean)	trees built by System F programs (System FT)	impredicative trees
36 (Lean)	ladder programs build the next ladder's tree, iterated along a tree	self-defined ladder lengths
37 (Lean)	tree languages stacked over the ladder jump	transfinite iterates of the jump
38 (Lean)	fast-growing hierarchy along Contender 37's tree	$h_{\mathrm{jumpTree}(n)}$ over F_{37}
39 (Lean)	languages over jump and collapse, depth F_{38}	h_{mixTree} over F_{38}
40 (Lean)	FT2 jump (tree recursion) under metalanguages	h_{metaTree} over F_{39}
41 (Lean)	ladder reader programs (FLT2) under metalanguages	$h_{\mathrm{readerTree}}$ over F_{40}
42 (Lean)	Buchholz collapse relative to the meta-jump	$h_{\mathrm{collapseTree}}$ over F_{41}
43 (Lean)	metalanguages whose closure collapses	$h_{\mathrm{collapseTree43}}$ over F_{42}
44 (Lean)	closures with tree-indexed relative collapse	$h_{\mathrm{levelTree44}}$ over F_{43}
45 (Lean)	the construction as its own base, iterated along a tree	$h_{\mathrm{thetaTree}}$ over F_{44}
46 (Lean)	universes of codes collapsed relative to the operator	$h_{\mathrm{univTree46}}$ over F_{45}
47 (Lean)	the self-based iteration, iterated along a tree	h_{psiTree} over F_{46}
48 (Lean)	tree-indexed hierarchy of iteration levels	$h_{\mathrm{levelIterTree}}$ over F_{47}
49 (Lean)	FT2 programs of type nat reading the tree, as the hierarchy's base	h_T over FT2 reflection on T
50 (Lean)	numeric ladder reader programs as the hierarchy's base	h_T over ladder reflection on T
51 (Lean)	System T over trees with the iteration operators as primitives	ladder reflection along the pro
52 (Lean)	the languages of Contender 51 stacked along a tree	ladder reflection along the sta
53 (Lean)	ladder reflection iterated along the tree	hyperreflection along the stack

For Contenders 15–37 the informal size describes the atom order. The ladder values see only its finite truncation (Section 38).